

Reducing Downtime in Manufacturing: Causes & Solutions

Strategies to Improve Efficiency & Productivity

Google Data Analyst Specialist
CAI2_DAT1_G5



Our team:



Mohamed Adel



Mohamed Gamal



Peter Mohareb



Eng/ Ahmed Samir

Agenda



Introduction

Data Loading

Data Cleaning & Preprocessing

Data Transformation

Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

Recommendations





Introduction



Data Loading

Data Cleaning & Preprocessing

Data Transformation

Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

Recommendations

INTRO-
DUCT-
ION



• What is Downtime in Manufacturing?

Unplanned stoppages in production due to equipment failure, maintenance, or inefficiencies.

Downtime in manufacturing refers to periods when production stops. It includes planned (maintenance, changeovers) and unplanned (breakdowns, shortages) stoppages.

Downtime reduces efficiency, increases costs, and delays deliveries.



• *Manufacturing Downtime Factors:*

1. Emergency stop.
2. Batch change
3. Labeling error
4. Inventory shortage
5. Product spill
6. Machine adjustment
7. Machine failure
8. Batch coding error
9. Conveyor belt jam
10. Calibration error
11. Label switch
12. Other



Categorization of Factors:

Non-Operator

Emergency Stop
Labeling error
Inventory shortage
Machine failure
Conveyor belt jam
Other

Operator

Batch change
Product spill
Machine adjustment
Batch coding error
Calibration error
Label switch

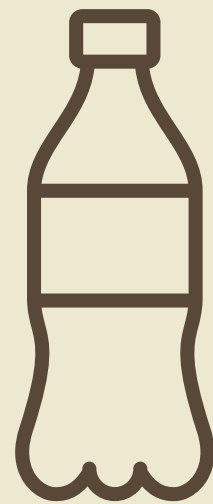
Numbers and Names of Operators:

There are 4 operators:

- Mac
- Dennis
- Charlie
- Dee



Products



Product	Flavor	Size
OR-600	Orange	600 ml
LE-600	Lemon lime	600 ml
CO-600	Cola	600 ml
DC-600	Diet Cola	600 ml
RB-600	Root Berry	600 ml
CO-2L	Cola	2 L

Data Overview:

Workdays = 5

Batches = 38

Products = 6

Total Production time = 3858 min

Factors = 12

Downtime = 1388 min

6 Factors
Non-Operator
(Machine)

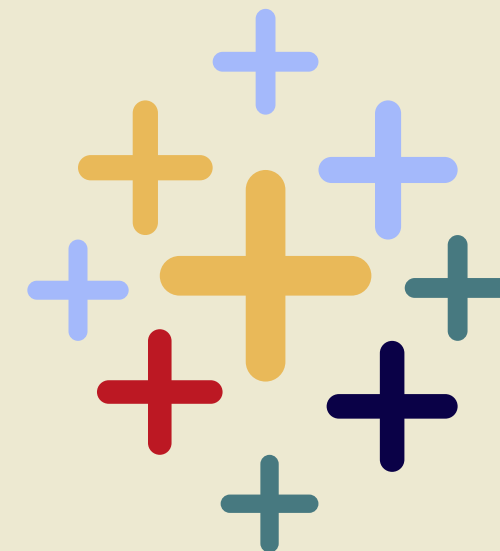
612 min

6 Factors
Operator
(Human)

776 min



Tools Used:



LOADING



Introduction



Data Loading



Data Cleaning & Preprocessing

Data Transformation

Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

Recommendations

- Using python

```
##Set Up Your Environment  
##Make sure you have the required libraries installed:  
##!pip install pandas matplotlib openpyxl statsmodels scikit-learn
```

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
from openpyxl import load_workbook  
from sklearn.linear_model import LinearRegression
```

```
##Phase 1: Build Data Model, Data Cleaning and Preprocessing
```

```
##Step 1: Load the Data
```

```
file_path = r"F:\Rawad Misr\Data Analysis\Projects\Final Projects\Final Project\Manufacturing_Line_Productivity.xlsx"  
sheets_dict = pd.read_excel(file_path, sheet_name=None) # Read all sheets
```

```
# Check the keys (sheet names)  
print(sheets_dict.keys())
```

```
Line_downtime = sheets_dict["Line downtime"]  
df_productivity = sheets_dict["Line productivity"]  
df_Products = sheets_dict["Products"]  
Downtime_factors = sheets_dict["Downtime factors"]
```

```
dict_keys(['Line productivity', 'Products', 'Downtime factors', 'Line downtime'])
```

```
# Set display options to show all rows & columns  
pd.set_option("display.max_rows", None) # Show all rows  
pd.set_option("display.max_columns", None) # Show all columns  
pd.set_option("display.expand_frame_repr", False) # Prevent column wrapping
```

```
# Loop through each sheet and display its full content  
for sheet_name, df in sheets_dict.items():  
    print(f"\n • Sheet: {sheet_name}\n")
```

```
# Display DataFrame info before full content  
print("\n🔍 Data Info:")  
df.info()
```

```
print("=" * 100) # Separator for readability  
# Display full DataFrame  
print(df)
```

• Using SQL

```
= create database Manufacturing_Downtime
-
= create table Line_productivity
  (Date date not null,
   Product varchar(50) not null,
   Batch int not null,
   Operator varchar(50) not null,
   Start_Time time not null,
   End_Time time not null)
-
= create table Products
  (Product varchar(50) not null,
   Flavor varchar(50) not null,
   Size varchar(50) not null,
   Min_batch_time int not null)
-
= create table Downtime_factors
  (Factor int not null,
   Description varchar(100) not null,
   Operator_Error varchar(50) not null)
-
= create table Line_downtime_factor1
  (Batch int not null,
   Factor int not null,
   Batch_downtime_by_factor int)
```

```
create table Line_downtime_factor
  (Batch int not null,
   one int,
   two int,
   three int,
   four int,
   five int,
   six int,
   seven int,
   eight int,
   nine int,
   ten int,
   eleven int,
   twelve int)
```

```
USE Manufacturing_Downtime;
GO
= BULK INSERT Line_productivity
FROM 'D:\Data Analyst\Project DEPI\SQL\Line productivity.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2
);
= BULK INSERT Products
FROM 'D:\Data Analyst\Project DEPI\SQL\Products.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2
);
= BULK INSERT Downtime_factors
FROM 'D:\Data Analyst\Project DEPI\SQL\Downtime factors.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2
);
= BULK INSERT Line_downtime_factor
FROM 'D:\Data Analyst\Project DEPI\SQL\Line downtime.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2
);
= BULK INSERT Line_downtime_factor1
FROM 'D:\Data Analyst\Project DEPI\SQL\Line downtime 1.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2
);
```

Introduction

Data Loading

✓ *Data Cleaning & Preprocessing* ✓

Data Transformation

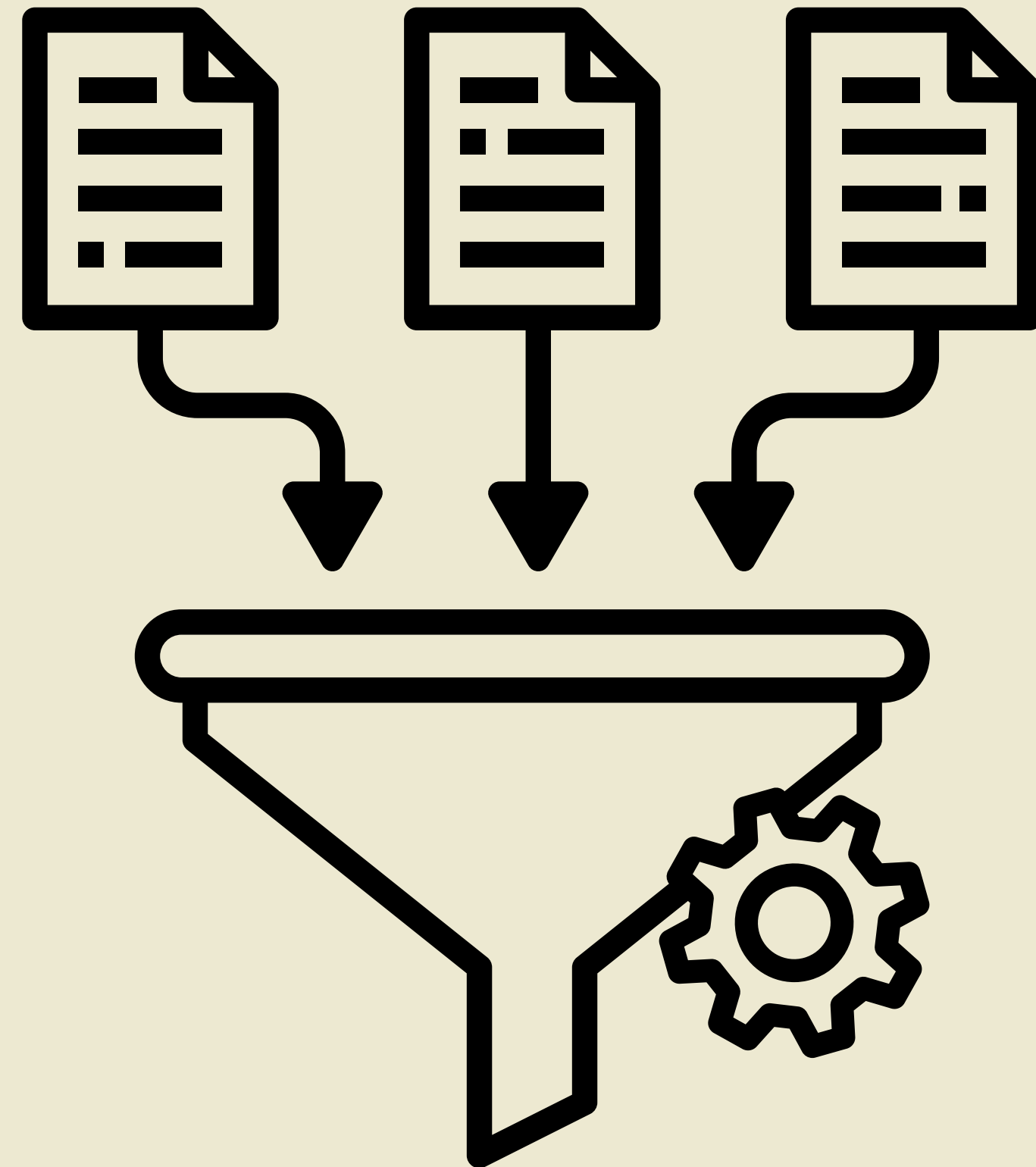
Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

Recommendations



- Handle missing values.
- Remove duplicates.
- Correct inconsistencies and errors.
- Convert data types as needed.

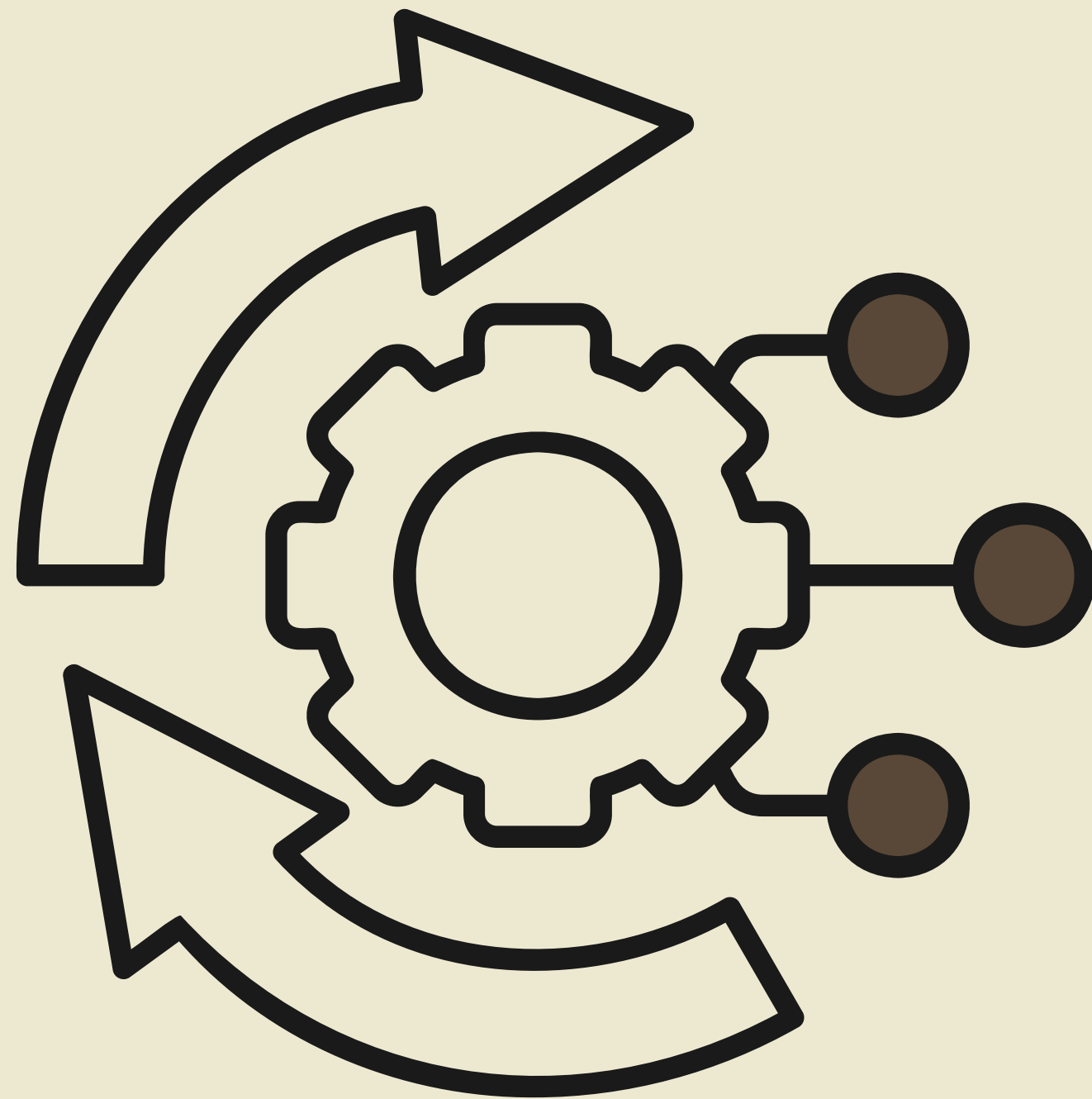
```
update Line_downtime_factor
set
    one=coalesce(one , 0 ),
    two=coalesce(two , 0 ),
    three=coalesce(three , 0 ),
    four=coalesce(four , 0 ),
    five=coalesce(five , 0 ),
    six=coalesce(six , 0 ),
    seven=coalesce(seven , 0 ),
    eight=coalesce(eight , 0 ),
    nine=coalesce(nine , 0 ),
    ten=coalesce(ten , 0 ),
    eleven=coalesce(eleven , 0 ),
    twelve=coalesce(twelve , 0 )
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Batch	1	2	3	4	5	6	7	8	9	10	11	12	Total Downtime by Batch
2	422111	0	60	0	0	0	0	15	0	0	0	0	0	75
3	422112	0	20	0	0	0	0	0	20	0	0	0	0	40
4	422113	0	50	0	0	0	0	0	0	0	0	0	0	50
5	422114	0	0	0	25	0	15	0	0	0	0	0	0	40
6	422115	0	0	0	0	0	0	0	0	0	24	0	0	24
7	422116	0	0	0	0	0	0	0	0	0	0	0	0	0
8	422117	0	0	0	0	0	0	0	0	0	0	0	0	0

```
# Replace NaN values with 0
Line_downtime.fillna(0, inplace=True)

Line_downtime.head()
```

	Unnamed: 0	Downtime factor	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10	Unnamed: 11	Unnamed: 12
0	Batch	1.0	2.0	3.0	4.0	5.0	6.0	7.0	8.0	9.0	10.0	11.0	12.0
1	422111	0.0	60.0	0.0	0.0	0.0	0.0	15.0	0.0	0.0	0.0	0.0	0.0
2	422112	0.0	20.0	0.0	0.0	0.0	0.0	0.0	20.0	0.0	0.0	0.0	0.0
3	422113	0.0	50.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	422114	0.0	0.0	0.0	25.0	0.0	15.0	0.0	0.0	0.0	0.0	0.0	0.0



Introduction

Data Loading

Data Cleaning & Preprocessing



Data Transformation



Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

Recommendations

• Creating an unpivot column for distribution of downtime to each factor.

```
create table Line_downtime_factor1
(
  Batch int not null,
  Factor int not null,
  Batch_downtime_by_factor int)

BULK INSERT Line_downtime_factor1
FROM 'D:\Data Analyst\Project DEPI\SQL\Line downtime 1.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2)
```

```
# Read the sheet, skipping the first row
Line_downtime = pd.read_excel(file_path, sheet_name="Line downtime", header=1)

# Remove extra spaces from column names and ensure they are strings
Line_downtime.columns = Line_downtime.columns.astype(str).str.strip()

# Rename "Unnamed" downtime factor columns sequentially as "Factor 1" to "Factor 12"
for i, col in enumerate(Line_downtime.columns[1:], start=1): # Skipping first column (Batch)
    if "Unnamed" in col or pd.isna(col): # Handling NaN column names
        Line_downtime.rename(columns={col: f"Factor {i}"}, inplace=True)

# Identify the batch column (assuming it's the first column)
batch_col = Line_downtime.columns[0] # First column should be "Batch"

# Reshape the data using melt
df_melted = Line_downtime.melt(id_vars=[batch_col], var_name="Downtime Factor", value_name="Minutes")

# Fill missing "Downtime Factor" cells correctly
if df_melted["Downtime Factor"].isna().sum() > 0:
    df_melted["Downtime Factor"] = df_melted["Downtime Factor"].fillna(method="ffill")

# Remove rows where "Minutes" is NaN
df_melted.dropna(subset=["Minutes"], inplace=True)

# Convert "Batch" column to numeric type and sort from smallest to largest
df_melted.dropna(subset=[batch_col], inplace=True)
df_melted.sort_values(by=[batch_col], inplace=True)

# Reset index and remove it from display
df_melted.reset_index(drop=True, inplace=True)

# Display the transformed data
print(df_melted.to_string(index=False)) # Removing index from output
```

Batch	Downtime Factor	Minutes
422111	2	60.0
422111	7	15.0
422112	2	20.0
422112	8	20.0
422113	2	50.0
422114	6	15.0
422114	4	25.0
422115	10	24.0
422117	2	10.0
422117	6	5.0
422118	6	14.0
422118	11	10.0
422118	7	16.0
422118	12	20.0
422119	4	25.0
422120	4	20.0
422120	9	17.0
422120	5	15.0
422121	7	15.0
422122	7	25.0
422123	4	43.0
422123	7	30.0
422124	5	20.0
422124	6	20.0
422125	11	10.0
422125	12	10.0
422126	8	44.0
422127	6	23.0
422128	5	22.0
422128	7	30.0
422129	12	15.0
422130	2	20.0
422131	10	10.0
422131	4	20.0
422133	7	20.0
422134	7	30.0
422134	8	20.0
422135	4	30.0
422135	12	15.0
422137	10	15.0

File		Home		Insert		Page Layout		Formulas	
Paste		Cut Copy Format Painter		Clipboard		Calibri 11 B I U Font			
SECURITY WARNING External Data Connections have									
J8									
	A	B	C						
1	Batch	Factor	Batch downtime by factor						
2	422111	2	60						
3	422111	7	15						
4	422112	2	20						
5	422112	8	20						
6	422113	2	50						
7	422114	4	25						
8	422114	6	15						
9	422115	10	24						
10	422117	2	10						
11	422117	6	5						
12	422118	6	14						
13	422118	7	16						
14	422118	11	10						
15	422118	12	20						
16	422119	4	25						
17	422120	4	20						
18	422120	5	15						
19	422120	9	17						
20	422121	7	15						
21	422122	7	25						
22	422123	4	43						
23	422123	7	30						
24	422124	5	20						
25	422124	6	20						
26	422125	11	10						
		Line downtime		Sheet1					
Ready									

- Add calculated column that substrate End time from start time.

```
ALTER TABLE Line_productivity
ADD Duration_min AS
    DATEDIFF(MINUTE,CASE(CAST(CONVERT(VARCHAR, Date, 23) + ' ' + CONVERT(VARCHAR, Start_Time, 8) AS DATETIME2),
    WHEN End_Time < Start_Time
        THEN CAST(CONVERT(VARCHAR, DATEADD(DAY, 1, Date), 23) + ' ' + CONVERT(VARCHAR, End_Time, 8) AS DATETIME2)
        ELSE CAST(CONVERT(VARCHAR, Date, 23) + ' ' + CONVERT(VARCHAR, End_Time, 8) AS DATETIME2)
    END)
```

```
# Convert "Start Time" and "End Time" to datetime format, keeping NaT for missing values
df_productivity["Start Time"] = pd.to_datetime(df_productivity["Start Time"], format="%H:%M:%S", errors="coerce")
df_productivity["End Time"] = pd.to_datetime(df_productivity["End Time"], format="%H:%M:%S", errors="coerce")

# Calculate the time difference (Batch Duration) in minutes
df_productivity["Batch Duration"] = (df_productivity["End Time"] - df_productivity["Start Time"]).dt.total_seconds() / 60

# Handle missing values
df_productivity.loc[df_productivity["End Time"].isna(), "Batch Duration"] = None
df_productivity.loc[df_productivity["Start Time"].isna(), "Batch Duration"] = None

# Handle cases where End Time is before Start Time (e.g., shift past midnight)
df_productivity["Batch Duration"] = df_productivity["Batch Duration"].apply(lambda x: x + 1440 if x < 0 else x)

# Convert "Start Time" and "End Time" back to string format to avoid date display in Excel
df_productivity["Start Time"] = df_productivity["Start Time"].dt.strftime("%H:%M:%S")
df_productivity["End Time"] = df_productivity["End Time"].dt.strftime("%H:%M:%S")

# Ensure "Batch Duration" remains numeric to prevent Excel from treating it as a date
df_productivity["Batch Duration"] = df_productivity["Batch Duration"].astype(float)

# ✅ Write the transformed data to a new sheet without modifying "Line productivity"
with pd.ExcelWriter(file_path, engine="openpyxl", mode="a") as writer:
    df_productivity.to_excel(writer, sheet_name="Transformed Line Productivity", index=False, float_format="%0.2f")

# ✅ Display the transformed DataFrame
print(df_productivity)

print("Sheet 'Transformed Line Productivity' created successfully!")
```

	Date	Product	Batch	Operator	Start Time	End Time	Batch Duration
0	2024-08-29	OR-600	422111	Mac	11:50:00	14:05:00	135.0
1	2024-08-29	LE-600	422112	Mac	14:05:00	15:45:00	100.0
2	2024-08-29	LE-600	422113	Mac	15:45:00	17:35:00	110.0
3	2024-08-29	LE-600	422114	Mac	17:35:00	19:15:00	100.0
4	2024-08-29	LE-600	422115	Charlie	19:15:00	20:39:00	84.0
5	2024-08-29	LE-600	422116	Charlie	20:39:00	21:39:00	60.0
6	2024-08-29	LE-600	422117	Charlie	21:39:00	22:54:00	75.0
7	2024-08-30	CO-600	422118	Dee	04:05:00	06:05:00	120.0
8	2024-08-30	CO-600	422119	Dee	06:05:00	07:30:00	85.0
9	2024-08-30	CO-600	422120	Dee	07:30:00	09:22:00	112.0
10	2024-08-30	CO-600	422121	Dennis	09:22:00	10:37:00	75.0
11	2024-08-30	CO-600	422122	Dennis	10:37:00	12:02:00	85.0
12	2024-08-30	CO-600	422123	Dennis	12:02:00	14:15:00	133.0
13	2024-08-30	CO-600	422124	Dennis	14:15:00	15:55:00	100.0
14	2024-08-30	CO-600	422125	Charlie	15:55:00	17:15:00	80.0
15	2024-08-30	CO-600	422126	Charlie	17:15:00	18:59:00	104.0
16	2024-08-30	CO-600	422127	Charlie	18:59:00	20:22:00	83.0
17	2024-08-30	CO-600	422128	Charlie	20:22:00	22:14:00	112.0
18	2024-08-30	CO-600	422129	Charlie	22:14:00	23:29:00	75.0
19	2024-08-31	CO-600	422130	Dee	07:45:00	09:05:00	80.0
20	2024-08-31	CO-600	422131	Dee	09:05:00	10:35:00	90.0
21	2024-08-31	CO-600	422132	Dee	10:35:00	11:35:00	60.0
22	2024-08-31	DC-600	422133	Dee	11:35:00	12:55:00	80.0
23	2024-08-31	DC-600	422134	Mac	12:55:00	14:45:00	110.0
24	2024-08-31	DC-600	422135	Mac	14:45:00	16:30:00	105.0
25	2024-08-31	DC-600	422136	Mac	16:30:00	17:30:00	60.0
26	2024-09-02	RB-600	422137	Dee	01:00:00	02:45:00	105.0
27	2024-09-02	RB-600	422138	Dee	02:45:00	04:05:00	80.0
28	2024-09-02	RB-600	422139	Dee	04:05:00	05:40:00	95.0
29	2024-09-02	RB-600	422140	Dee	05:40:00	07:43:00	123.0
30	2024-09-02	RB-600	422141	Dennis	07:43:00	08:50:00	67.0
31	2024-09-02	RB-600	422142	Dennis	08:50:00	10:20:00	90.0
32	2024-09-02	RB-600	422143	Dennis	10:20:00	12:18:00	118.0
33	2024-09-02	CO-2L	422144	Dennis	12:18:00	14:50:00	152.0
34	2024-09-02	CO-2L	422145	Charlie	14:50:00	16:50:00	120.0
35	2024-09-02	CO-2L	422146	Charlie	16:50:00	19:30:00	160.0
36	2024-09-02	CO-2L	422147	Charlie	19:30:00	22:55:00	205.0
37	2024-09-03	CO-2L	422148	Mac	00:05:00	02:15:00	130.0

Sheet 'Transformed Line Productivity' created successfully!

- Add a new column called a minimum batch time for each batch.

```
# Load the necessary sheets
df_productivity = pd.read_excel(file_path, sheet_name="Transformed Line Productivity")
df_products = pd.read_excel(file_path, sheet_name="Transformed Products")

# Ensure necessary columns exist
required_columns_productivity = ["Product"]
required_columns_products = ["Product", "Min batch time"]

for col in required_columns_productivity:
    if col not in df_productivity.columns:
        raise KeyError(f"❌ '{col}' column not found in 'Transformed Line Productivity' sheet.")

for col in required_columns_products:
    if col not in df_products.columns:
        raise KeyError(f"❌ '{col}' column not found in 'Products' sheet.")

# ✅ Merge "Min batch time" from "Transformed Products" into "Transformed Line Productivity"
df_productivity = df_productivity.merge(df_products[["Product", "Min batch time"]], on="Product", how="left")

# ✅ Save the updated sheet back to Excel, replacing the old one
with pd.ExcelWriter(file_path, engine="openpyxl", mode="a", if_sheet_exists="replace") as writer:
    df_productivity.to_excel(writer, sheet_name="Transformed Line Productivity", index=False)

# ✅ Display the updated DataFrame
print(df_productivity)
print("✅ 'Transformed Line Productivity' sheet updated successfully!")
```

	Date	Product	Batch	Operator	Start Time	End Time	Batch Duration	Min batch time
0	2024-08-29	OR-600	422111	Mac	11:50:00	14:05:00	135	60
1	2024-08-29	LE-600	422112	Mac	14:05:00	15:45:00	100	60
2	2024-08-29	LE-600	422113	Mac	15:45:00	17:35:00	110	60
3	2024-08-29	LE-600	422114	Mac	17:35:00	19:15:00	100	60
4	2024-08-29	LE-600	422115	Charlie	19:15:00	20:39:00	84	60
5	2024-08-29	LE-600	422116	Charlie	20:39:00	21:39:00	60	60
6	2024-08-29	LE-600	422117	Charlie	21:39:00	22:54:00	75	60
7	2024-08-30	CO-600	422118	Dee	04:05:00	06:05:00	120	60
8	2024-08-30	CO-600	422119	Dee	06:05:00	07:30:00	85	60
9	2024-08-30	CO-600	422120	Dee	07:30:00	09:22:00	112	60
10	2024-08-30	CO-600	422121	Dennis	09:22:00	10:37:00	75	60
11	2024-08-30	CO-600	422122	Dennis	10:37:00	12:02:00	85	60
12	2024-08-30	CO-600	422123	Dennis	12:02:00	14:15:00	133	60
13	2024-08-30	CO-600	422124	Dennis	14:15:00	15:55:00	100	60
14	2024-08-30	CO-600	422125	Charlie	15:55:00	17:15:00	80	60
15	2024-08-30	CO-600	422126	Charlie	17:15:00	18:59:00	104	60
16	2024-08-30	CO-600	422127	Charlie	18:59:00	20:22:00	83	60
17	2024-08-30	CO-600	422128	Charlie	20:22:00	22:14:00	112	60
18	2024-08-30	CO-600	422129	Charlie	22:14:00	23:29:00	75	60
19	2024-08-31	CO-600	422130	Dee	07:45:00	09:05:00	80	60
20	2024-08-31	CO-600	422131	Dee	09:05:00	10:35:00	90	60
21	2024-08-31	CO-600	422132	Dee	10:35:00	11:35:00	60	60
22	2024-08-31	DC-600	422133	Dee	11:35:00	12:55:00	80	60
23	2024-08-31	DC-600	422134	Mac	12:55:00	14:45:00	110	60
24	2024-08-31	DC-600	422135	Mac	14:45:00	16:30:00	105	60
25	2024-08-31	DC-600	422136	Mac	16:30:00	17:30:00	60	60
26	2024-09-02	RB-600	422137	Dee	01:00:00	02:45:00	105	60
27	2024-09-02	RB-600	422138	Dee	02:45:00	04:05:00	80	60
28	2024-09-02	RB-600	422139	Dee	04:05:00	05:40:00	95	60
29	2024-09-02	RB-600	422140	Dee	05:40:00	07:43:00	123	60
30	2024-09-02	RB-600	422141	Dennis	07:43:00	08:50:00	67	60
31	2024-09-02	RB-600	422142	Dennis	08:50:00	10:20:00	90	60
32	2024-09-02	RB-600	422143	Dennis	10:20:00	12:18:00	118	60
33	2024-09-02	CO-2L	422144	Dennis	12:18:00	14:50:00	152	98
34	2024-09-02	CO-2L	422145	Charlie	14:50:00	16:50:00	120	98
35	2024-09-02	CO-2L	422146	Charlie	16:50:00	19:30:00	160	98
36	2024-09-02	CO-2L	422147	Charlie	19:30:00	22:55:00	205	98
37	2024-09-03	CO-2L	422148	Mac	00:05:00	02:15:00	130	98

✅ 'Transformed Line Productivity' sheet updated successfully!

- Add a new column called Total Downtime for each batch.

```
# Load the "Transformed Line Productivity" sheet
df_productivity = pd.read_excel(file_path, sheet_name="Transformed Line Productivity")

# ✅ Ensure "Batch Duration" and "Min batch time" are numeric
df_productivity["Batch Duration"] = pd.to_numeric(df_productivity["Batch Duration"], errors="coerce")
df_productivity["Min batch time"] = pd.to_numeric(df_productivity["Min batch time"], errors="coerce")

# ✅ Handle missing values before calculations
df_productivity["Batch Duration"] = df_productivity["Batch Duration"].fillna(0)
df_productivity["Min batch time"] = df_productivity["Min batch time"].fillna(0)

# ✅ Calculate "Total Downtime by Batch" (Minutes)
df_productivity["Total Downtime by Batch"] = df_productivity["Batch Duration"] - df_productivity["Min batch time"]

# ✅ Ensure "Total Downtime by Batch" has no NaN values
df_productivity["Total Downtime by Batch"] = df_productivity["Total Downtime by Batch"].fillna(0)

# ✅ Ensure "Batch Duration" is always positive (Handle negative values if needed)
df_productivity["Batch Duration"] = df_productivity["Batch Duration"].apply(lambda x: x + 1440 if x < 0 else x)

# ✅ Save back to "Transformed Line Productivity" without modifying other sheets
with pd.ExcelWriter(file_path, engine="openpyxl", mode="a", if_sheet_exists="replace") as writer:
    df_productivity.to_excel(writer, sheet_name="Transformed Line Productivity", index=False, float_format="%.2f")

# ✅ Display the updated DataFrame
print(df_productivity)
print("✅ 'Transformed Line Productivity' sheet updated successfully!")
```

	Date	Product	Batch	Operator	Start Time	End Time	Batch Duration	Min batch time	Total Downtime by Batch
0	2024-08-29	OR-600	422111	Mac	11:50:00	14:05:00	135	60	75
1	2024-08-29	LE-600	422112	Mac	14:05:00	15:45:00	100	60	40
2	2024-08-29	LE-600	422113	Mac	15:45:00	17:35:00	110	60	50
3	2024-08-29	LE-600	422114	Mac	17:35:00	19:15:00	100	60	40
4	2024-08-29	LE-600	422115	Charlie	19:15:00	20:39:00	84	60	24
5	2024-08-29	LE-600	422116	Charlie	20:39:00	21:39:00	60	60	0
6	2024-08-29	LE-600	422117	Charlie	21:39:00	22:54:00	75	60	15
7	2024-08-30	CO-600	422118	Dee	04:05:00	06:05:00	120	60	60
8	2024-08-30	CO-600	422119	Dee	06:05:00	07:30:00	85	60	25
9	2024-08-30	CO-600	422120	Dee	07:30:00	09:22:00	112	60	52
10	2024-08-30	CO-600	422121	Dennis	09:22:00	10:37:00	75	60	15
11	2024-08-30	CO-600	422122	Dennis	10:37:00	12:02:00	85	60	25
12	2024-08-30	CO-600	422123	Dennis	12:02:00	14:15:00	133	60	73
13	2024-08-30	CO-600	422124	Dennis	14:15:00	15:55:00	100	60	40
14	2024-08-30	CO-600	422125	Charlie	15:55:00	17:15:00	80	60	20
15	2024-08-30	CO-600	422126	Charlie	17:15:00	18:59:00	104	60	44
16	2024-08-30	CO-600	422127	Charlie	18:59:00	20:22:00	83	60	23
17	2024-08-30	CO-600	422128	Charlie	20:22:00	22:14:00	112	60	52
18	2024-08-30	CO-600	422129	Charlie	22:14:00	23:29:00	75	60	15
19	2024-08-31	CO-600	422130	Dee	07:45:00	09:05:00	80	60	20
20	2024-08-31	CO-600	422131	Dee	09:05:00	10:35:00	90	60	30
21	2024-08-31	CO-600	422132	Dee	10:35:00	11:35:00	60	60	0
22	2024-08-31	DC-600	422133	Dee	11:35:00	12:55:00	80	60	20
23	2024-08-31	DC-600	422134	Mac	12:55:00	14:45:00	110	60	50
24	2024-08-31	DC-600	422135	Mac	14:45:00	16:30:00	105	60	45
25	2024-08-31	DC-600	422136	Mac	16:30:00	17:30:00	60	60	0
26	2024-09-02	RB-600	422137	Dee	01:00:00	02:45:00	105	60	45
27	2024-09-02	RB-600	422138	Dee	02:45:00	04:05:00	80	60	20
28	2024-09-02	RB-600	422139	Dee	04:05:00	05:40:00	95	60	35
29	2024-09-02	RB-600	422140	Dee	05:40:00	07:43:00	123	60	63
30	2024-09-02	RB-600	422141	Dennis	07:43:00	08:50:00	67	60	7
31	2024-09-02	RB-600	422142	Dennis	08:50:00	10:20:00	90	60	30
32	2024-09-02	RB-600	422143	Dennis	10:20:00	12:18:00	118	60	58
33	2024-09-02	CO-2L	422144	Dennis	12:18:00	14:50:00	152	98	54
34	2024-09-02	CO-2L	422145	Charlie	14:50:00	16:50:00	120	98	22
35	2024-09-02	CO-2L	422146	Charlie	16:50:00	19:30:00	160	98	62
36	2024-09-02	CO-2L	422147	Charlie	19:30:00	22:55:00	205	98	107
37	2024-09-03	CO-2L	422148	Mac	00:05:00	02:15:00	130	98	32

✅ 'Transformed Line Productivity' sheet updated successfully!

Introduction

Data Loading

Data Cleaning & Preprocessing

Data Transformation

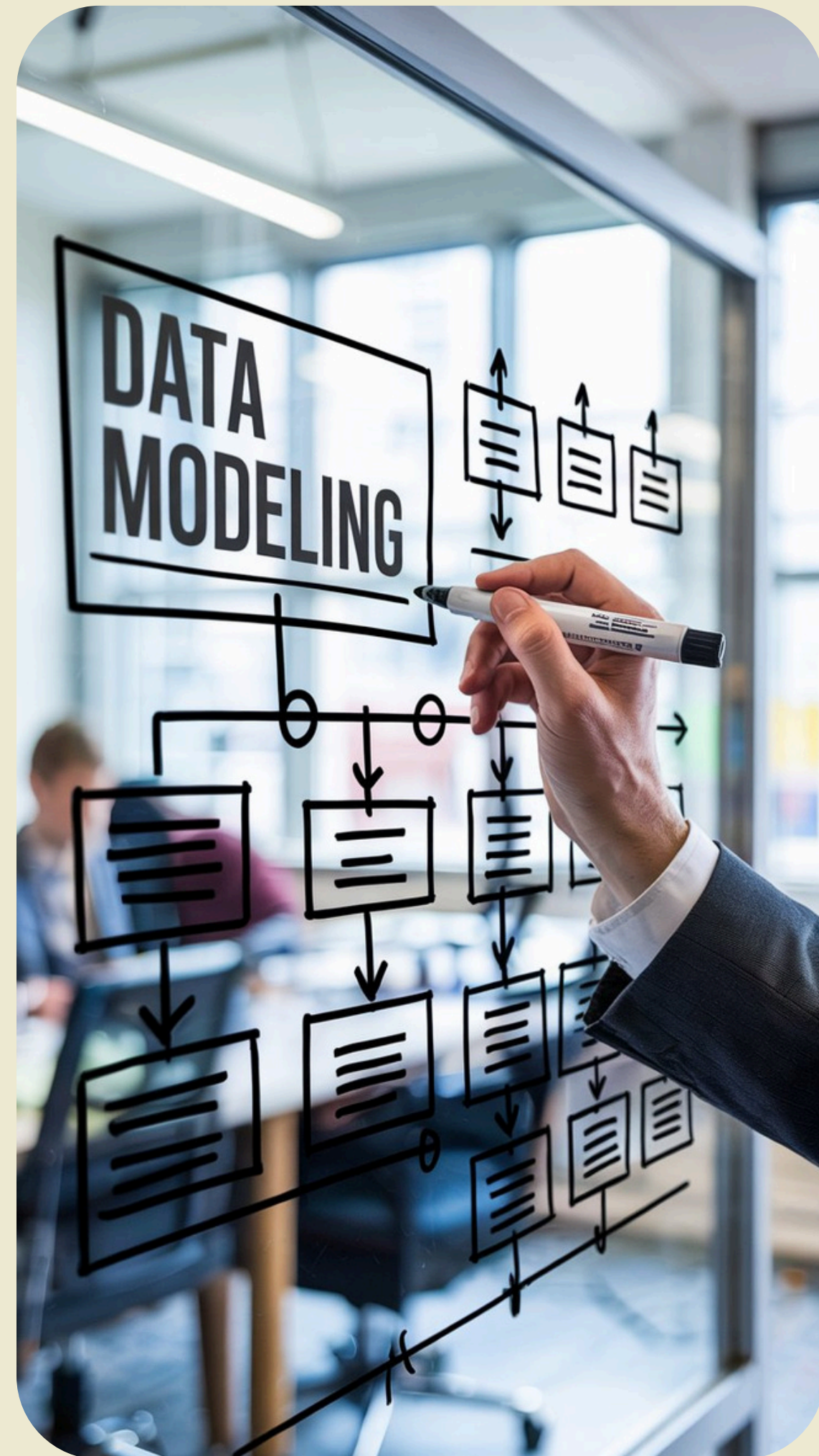
✓ *Data Modeling* ✓

Exploratory Data Analysis

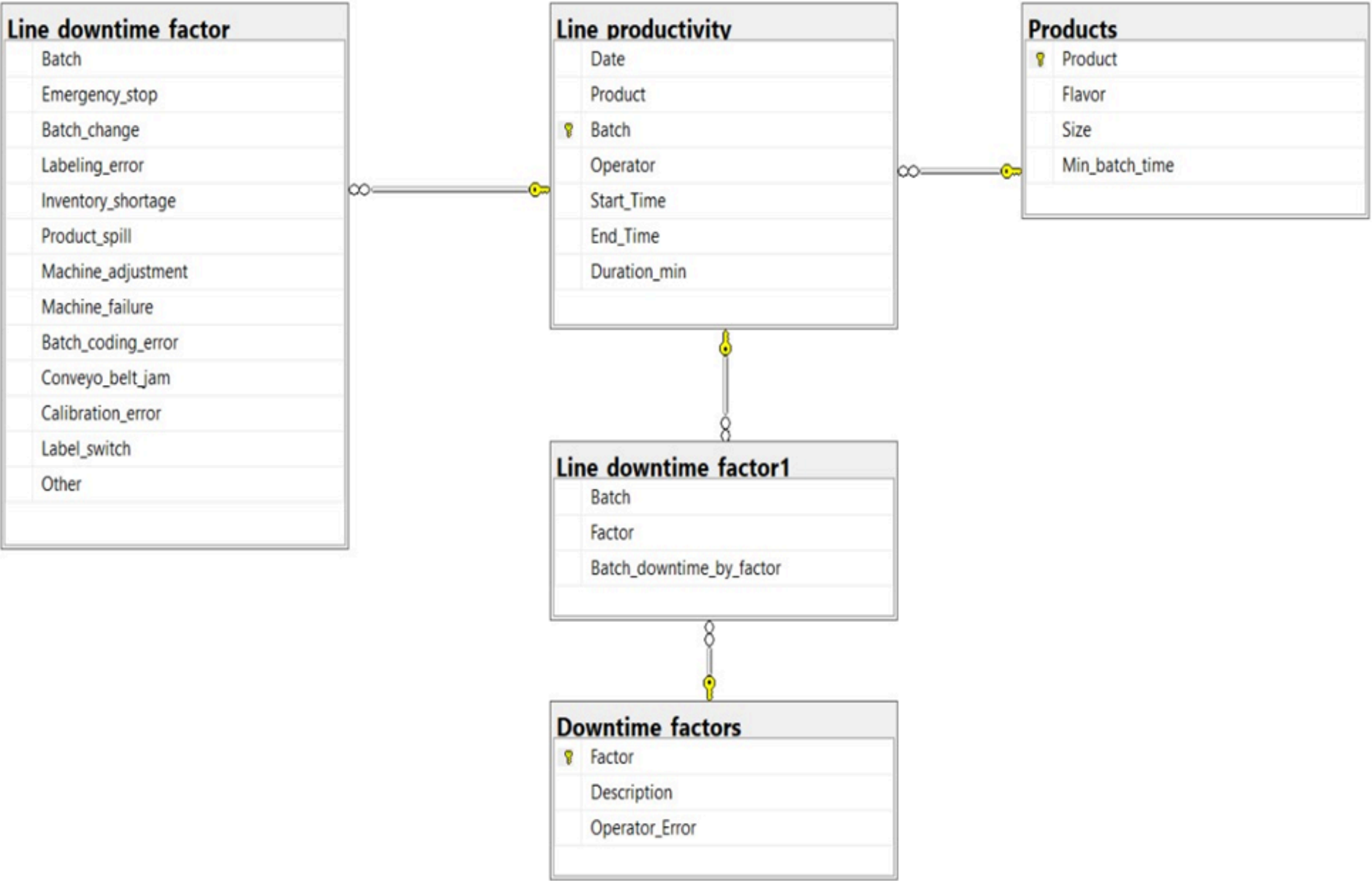
Dashboards

Forecasting

Recommendations



Data Modeling:





Introduction

Data Loading

Data Cleaning & Preprocessing

Data Transformation

Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

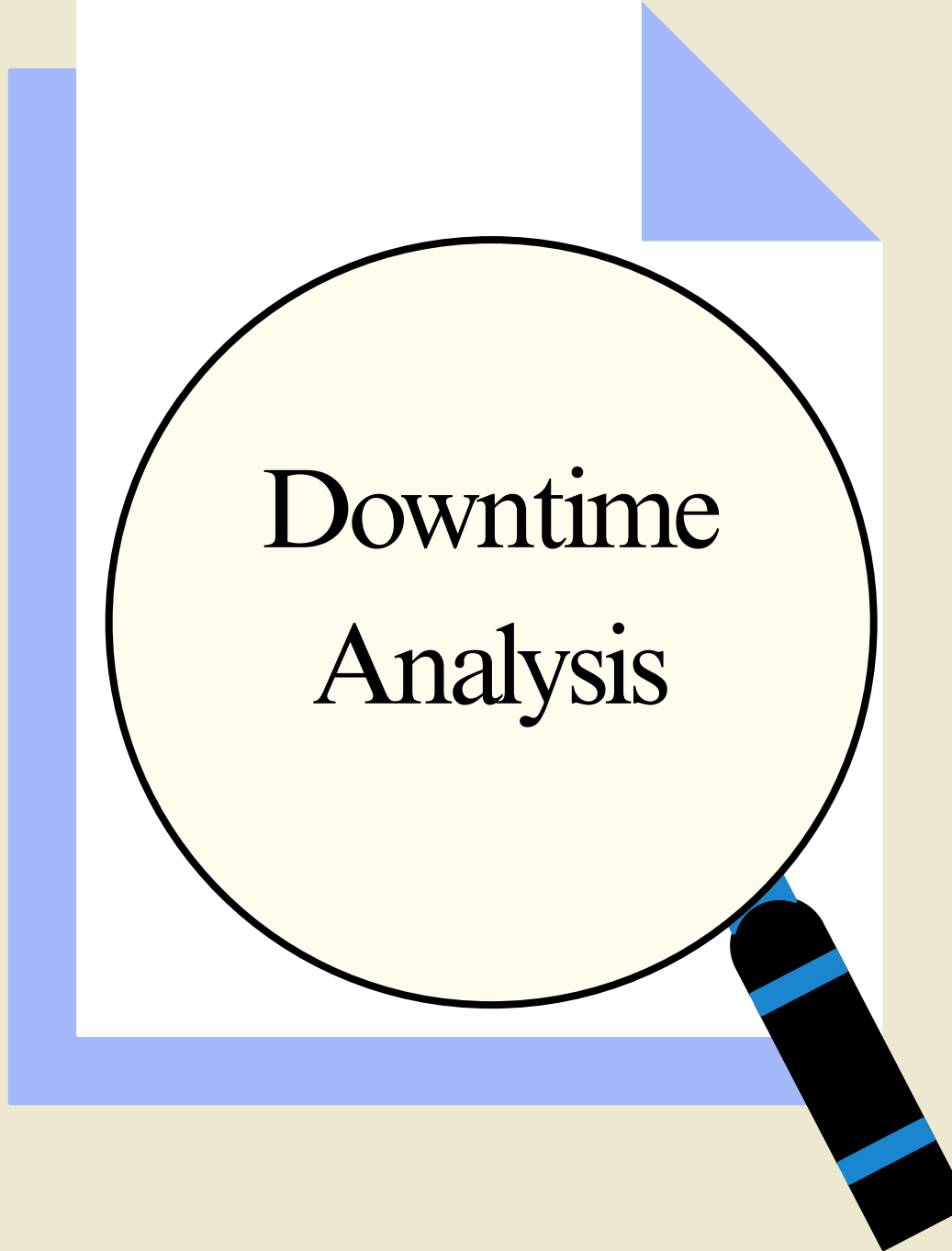
Recommendations



Exploratory Data Analysis

A graphic consisting of a white document with a folded top-right corner, a light blue border, and a large yellow circle in the center. A magnifying glass with a black handle and blue rings is positioned at the bottom right of the circle.

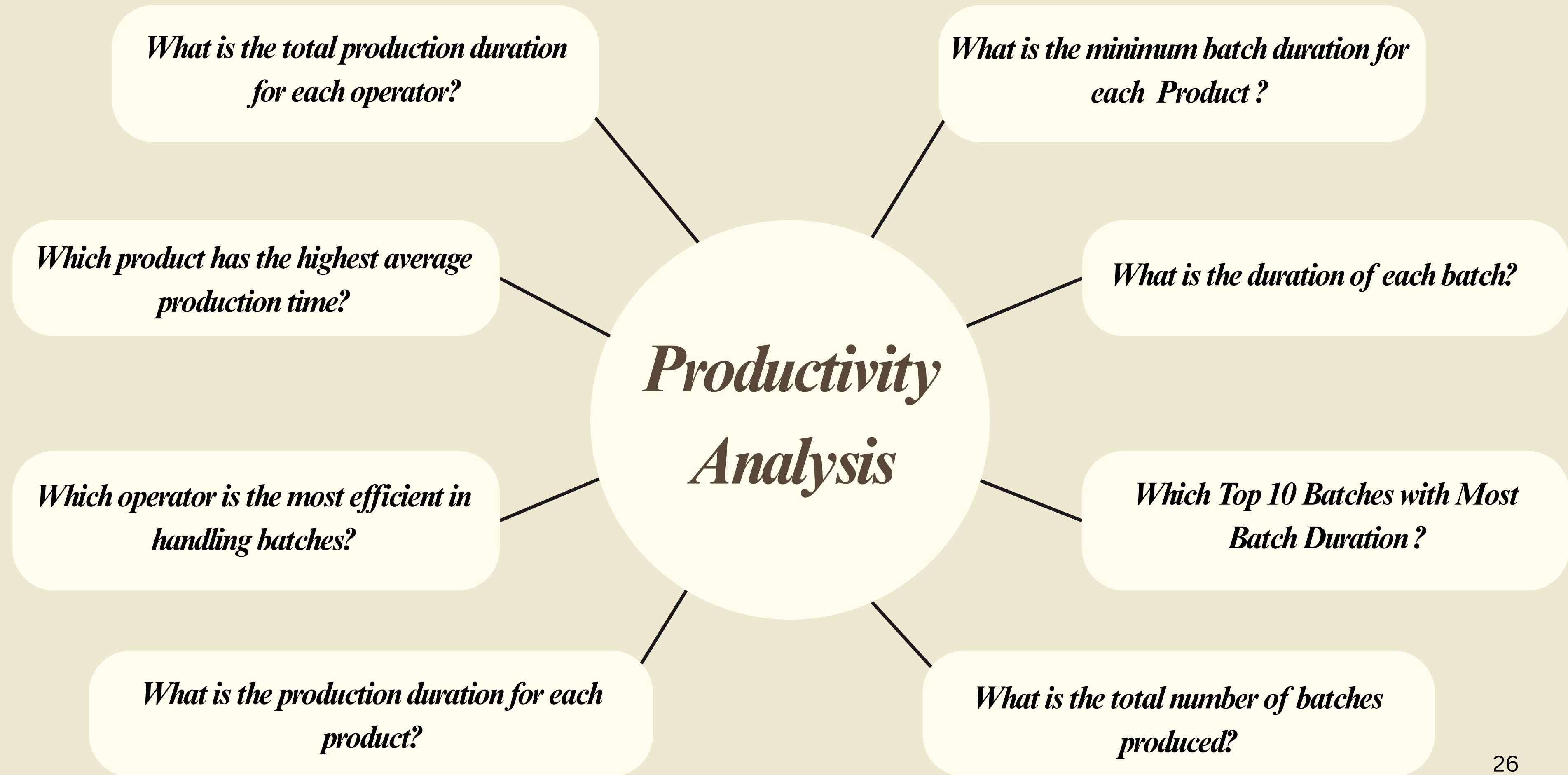
Productivity
Analysis

A graphic consisting of a white document with a folded top-right corner, a light blue border, and a large yellow circle in the center. A magnifying glass with a black handle and blue rings is positioned at the bottom right of the circle.

Downtime
Analysis

A graphic consisting of a white document with a folded top-right corner, a light blue border, and a large yellow circle in the center. A magnifying glass with a black handle and blue rings is positioned at the bottom right of the circle.

Performance
Trends



Downtime Analysis

```
graph TD; A((Downtime Analysis)) --- B(What are the most common downtime factors?); A --- C(How much downtime is caused by operator errors vs. other causes in percentage?); A --- D(What is the total downtime recorded?); A --- E(What is the total downtime for each day?); A --- F(Which Product experienced the most downtime?); A --- G(How much downtime is caused by operator errors vs. other causes in minutes?); A --- H(How many factors influence downtime, and what are they?); A --- I(How many factors influence downtime?)
```

What are the most common downtime factors?

How much downtime is caused by operator errors vs. other causes in percentage?

What is the total downtime recorded?

What is the total downtime for each day?

Which Product experienced the most downtime?

How much downtime is caused by operator errors vs. other causes in minutes?

How many factors influence downtime, and what are they?

How many factors influence downtime?

Performance Trends

Is there a relationship between batch size and downtime

Introduction

Data Loading

Data Cleaning & Preprocessing

Data Transformation

Data Modeling

Exploratory Data Analysis

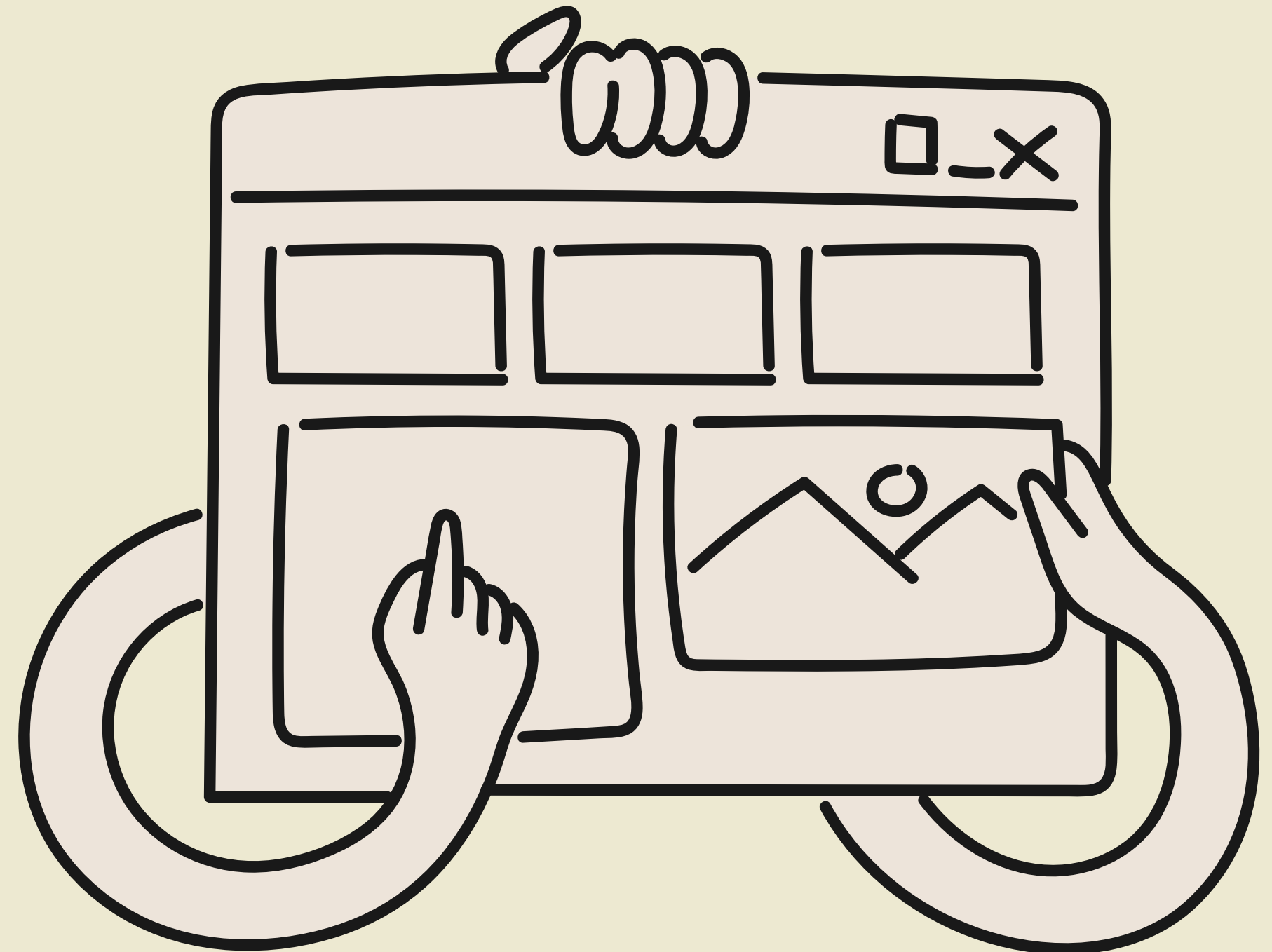


Dashboards



Forecasting

Recommendations

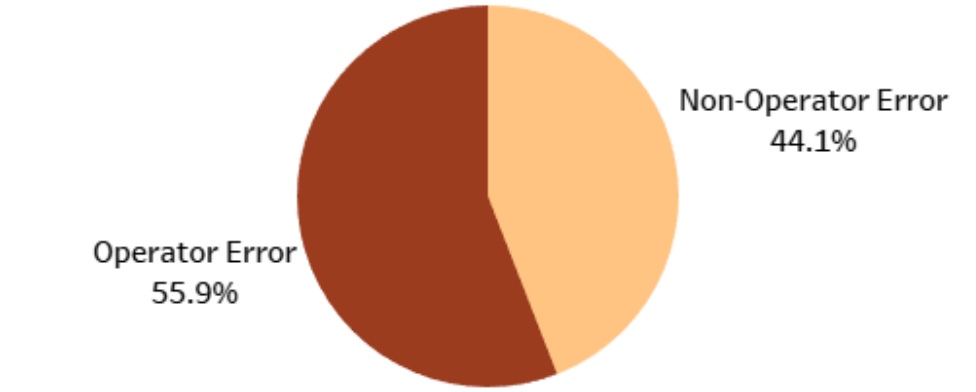


Downtime Analysis

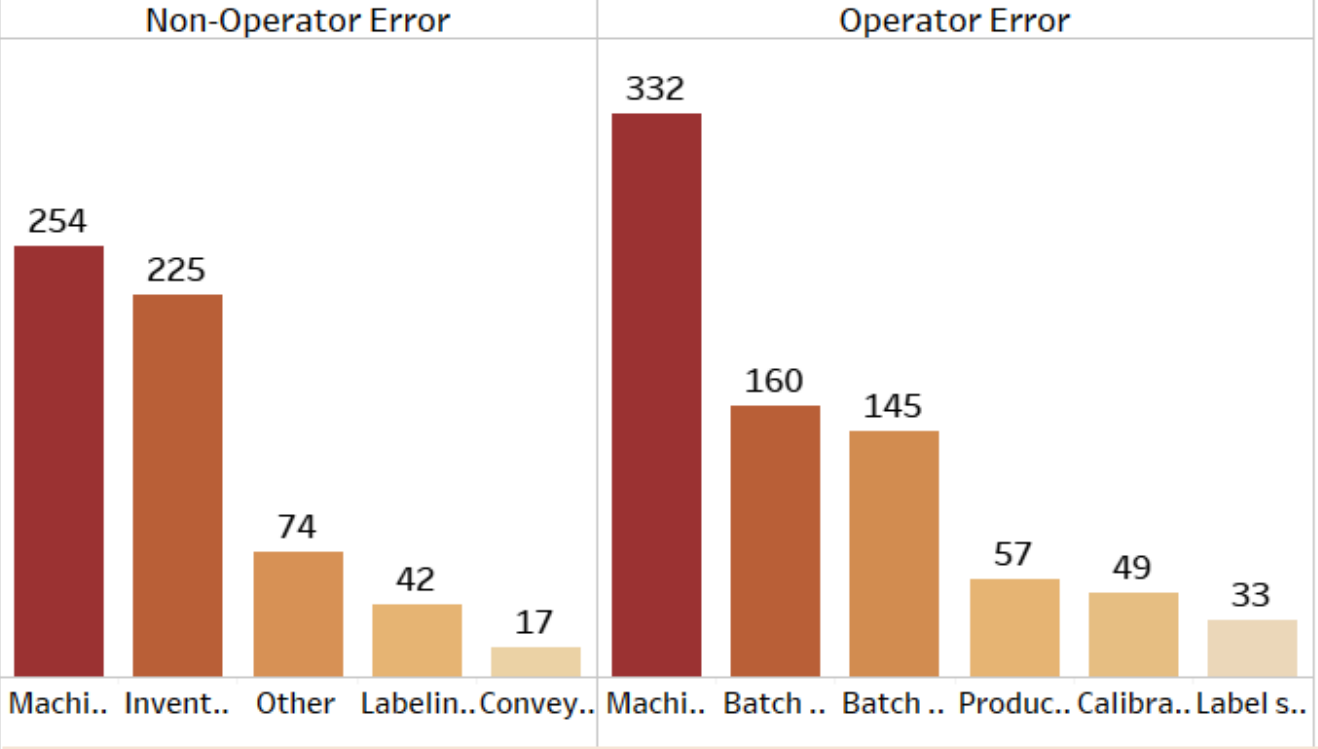
#Batchs	#Products	#Factors
38	6	12

Expected Prod. Time	Prod. Time	Downtime
2,470	3,858	1,388

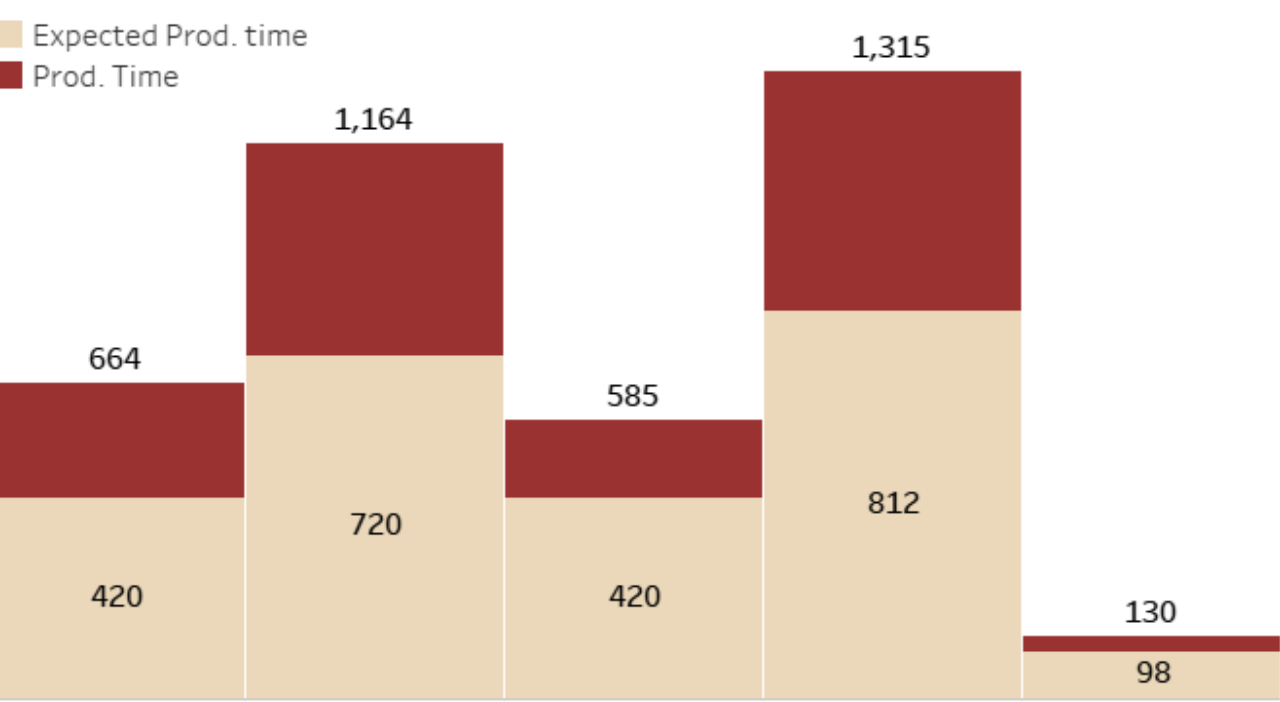
% of total Downtime per factor



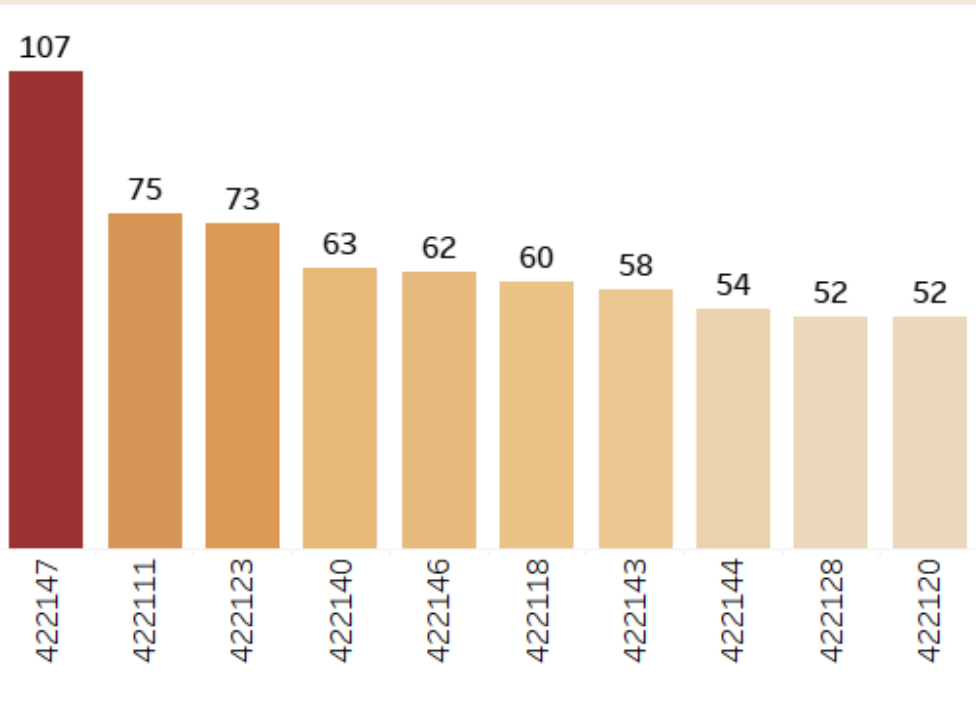
Total Downtime for each factor



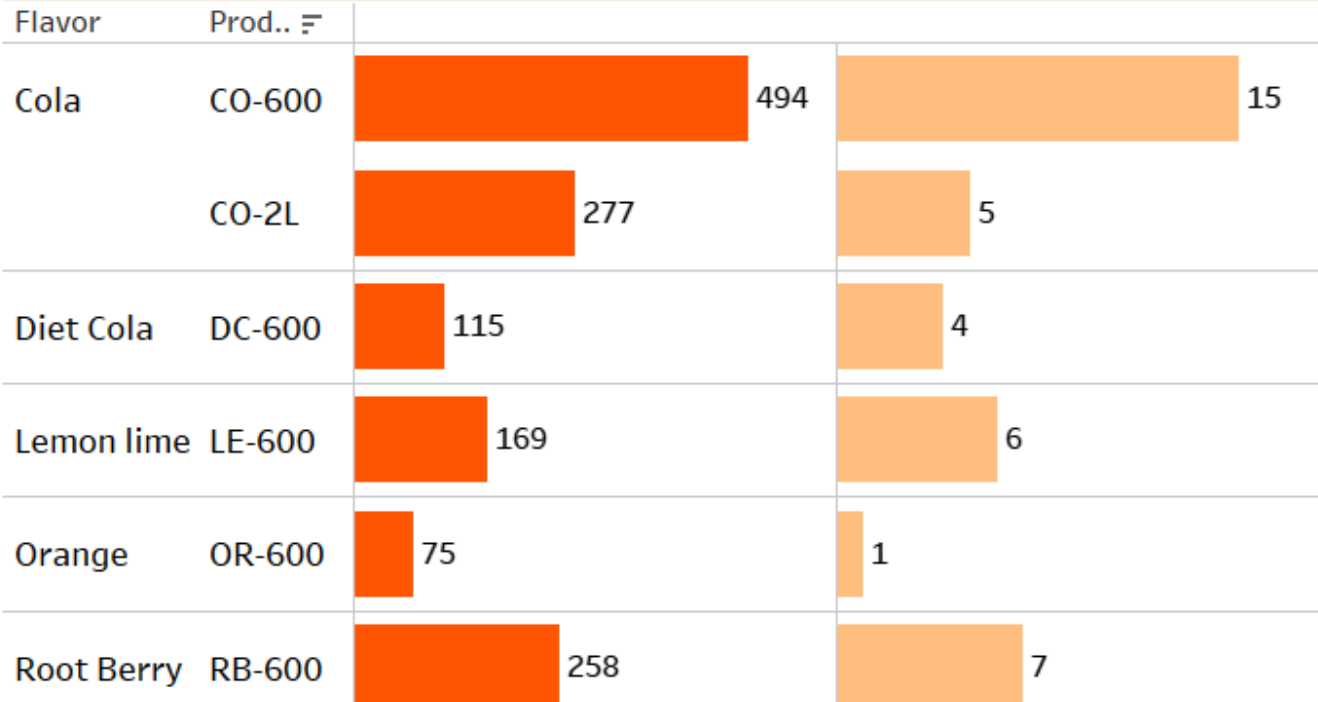
Expected VS Actual production duration



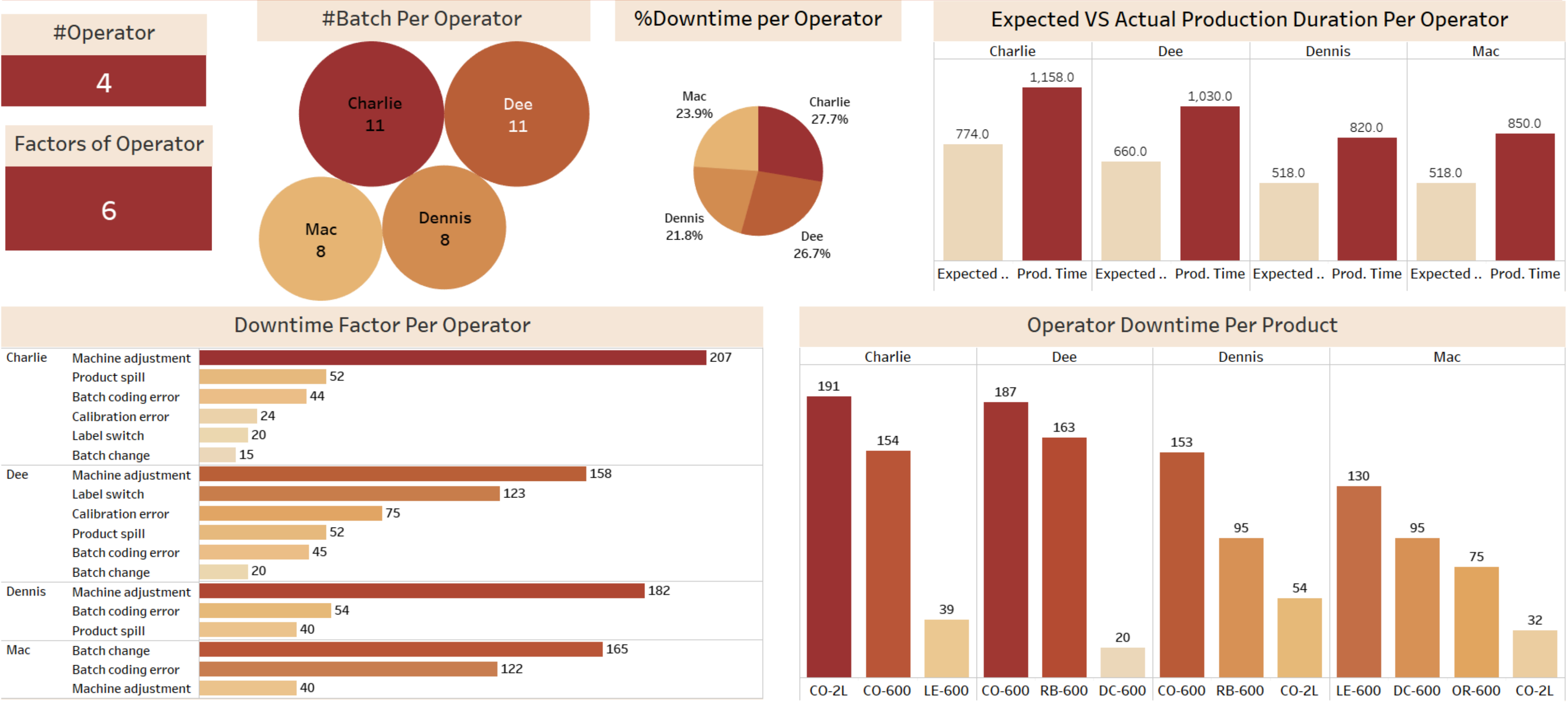
Top 10 Batches With Downtime



Downtime and Batches Per Product

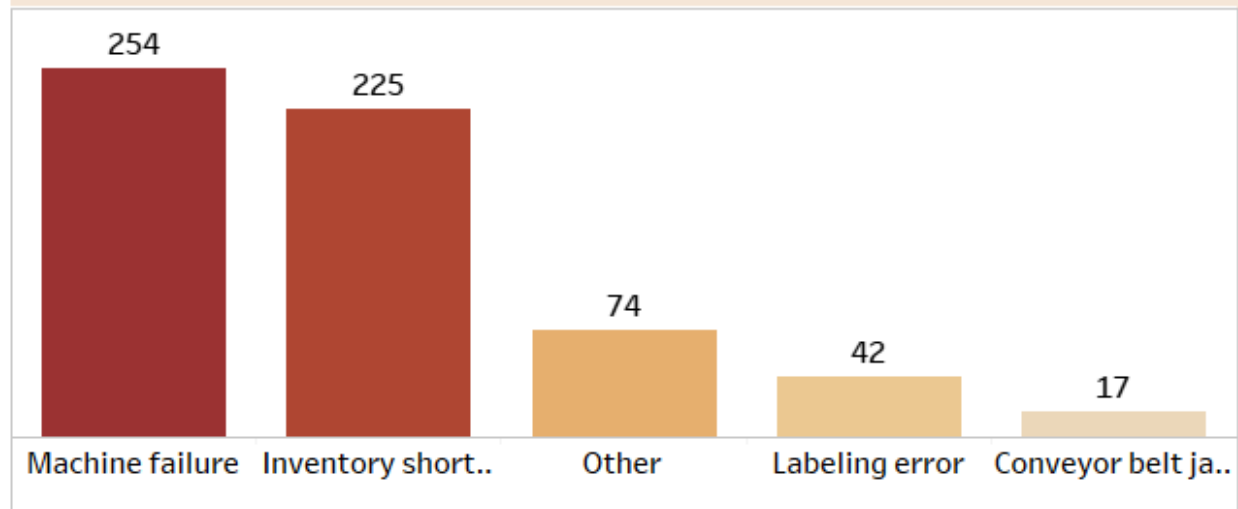


Downtime Operator Analysis

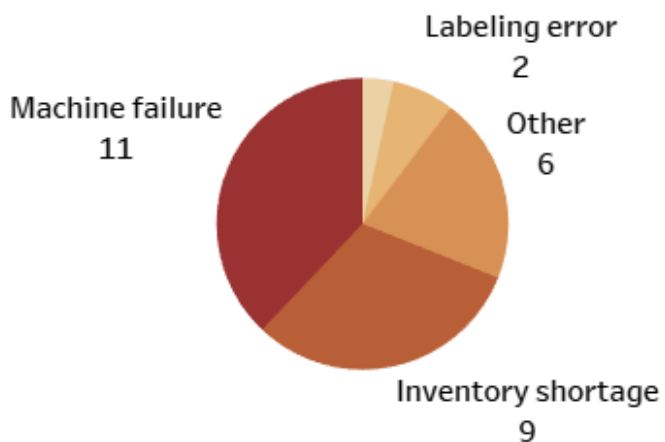


Downtime Non-Operator Analysis

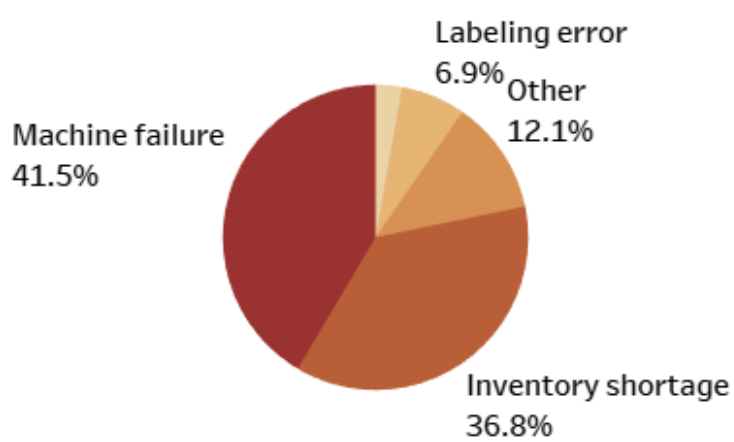
Downtime of factors



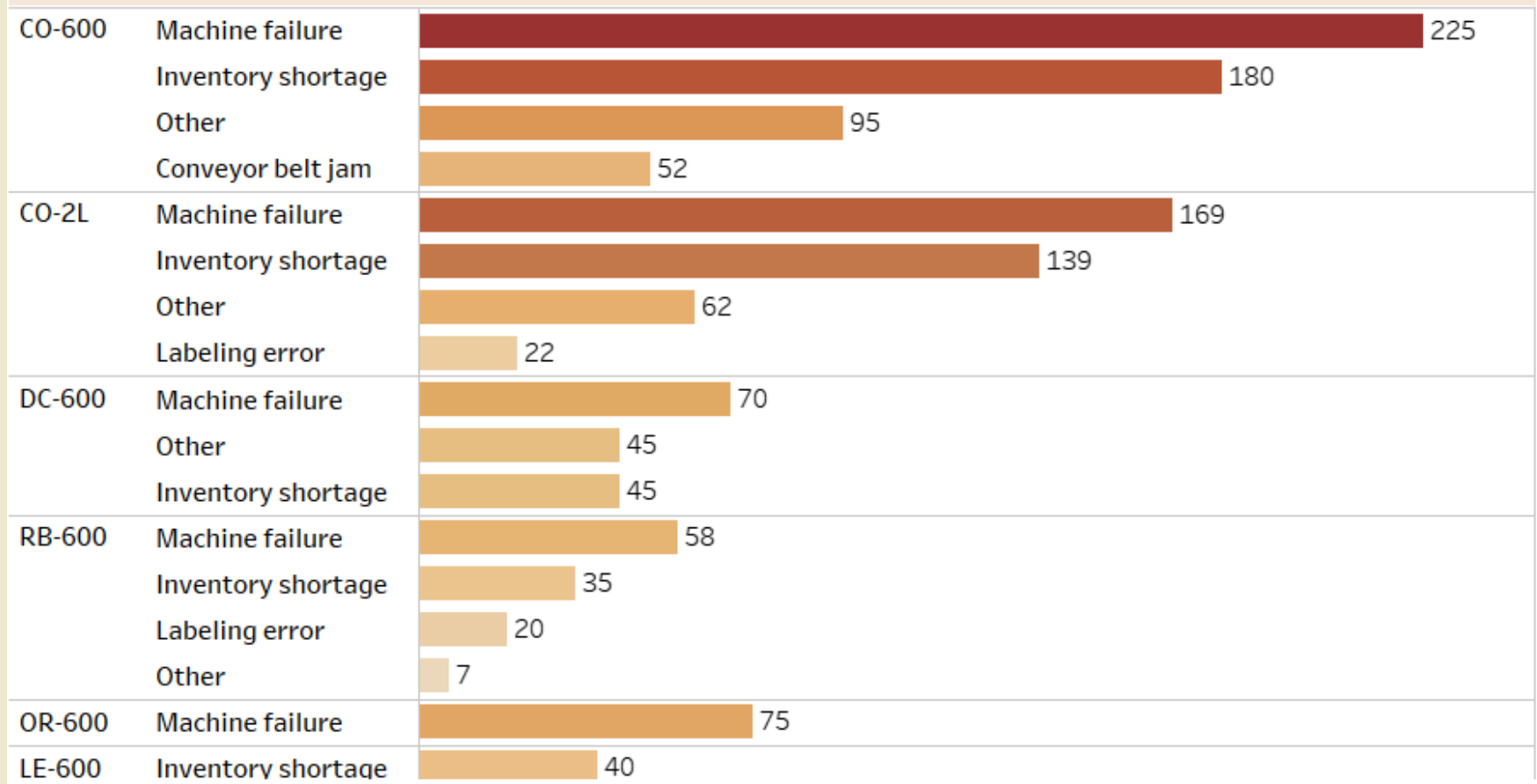
#Batch of Each Factor



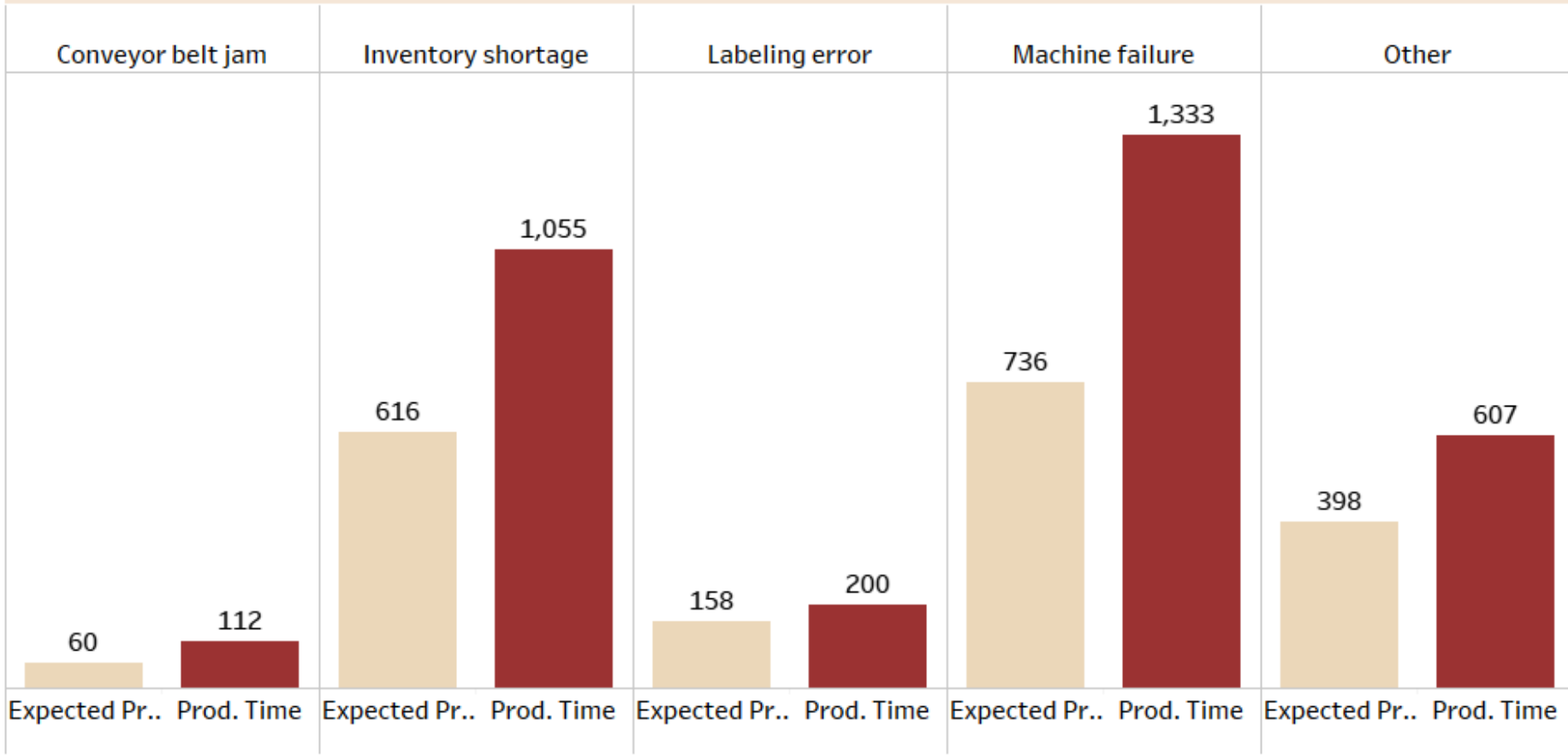
%Downtime of Each Factor



Downtime of factors for each Product



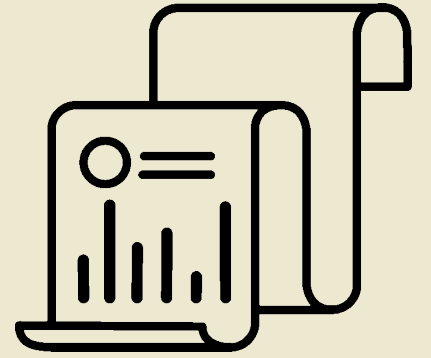
Expected VS Actual Production time





Introduction

Data Loading



Data Cleaning & Preprocessing

Data Transformation

Data Modeling

Exploratory Data Analysis

Dashboards



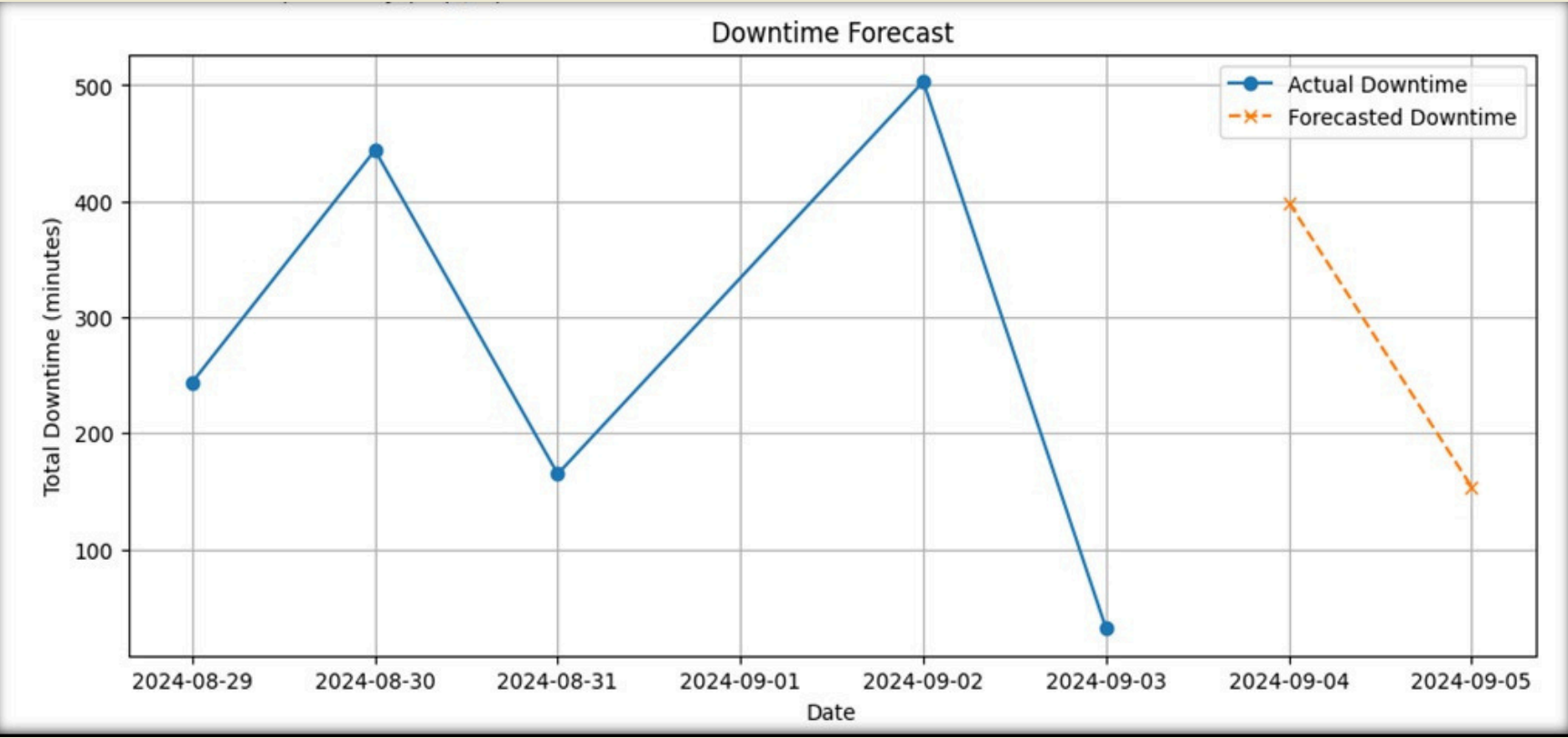
Forecasting



Recommendations

Forecasting:

- Prediction of downtime in the next day of operation.



```
# Forecast next two days
def forecast_next_two(model, last_values):
    pred_day1 = model.predict([last_values])[0]
    pred_day2 = model.predict([[pred_day1] + list(last_values[:2])])[0]
    return round(pred_day1), round(pred_day2)

last_downtime_vals = daily_summary['total_downtime'].iloc[-3:].values[::-1]
downtime_forecast = forecast_next_two(downtime_model, last_downtime_vals)

last_batch_vals = daily_summary['batch_count'].iloc[-3:].values[::-1]
batch_forecast = forecast_next_two(batch_model, last_batch_vals)

print("Predicted Downtime (next 2 days):", downtime_forecast)
print("Predicted Batch Count (next 2 days):", batch_forecast)

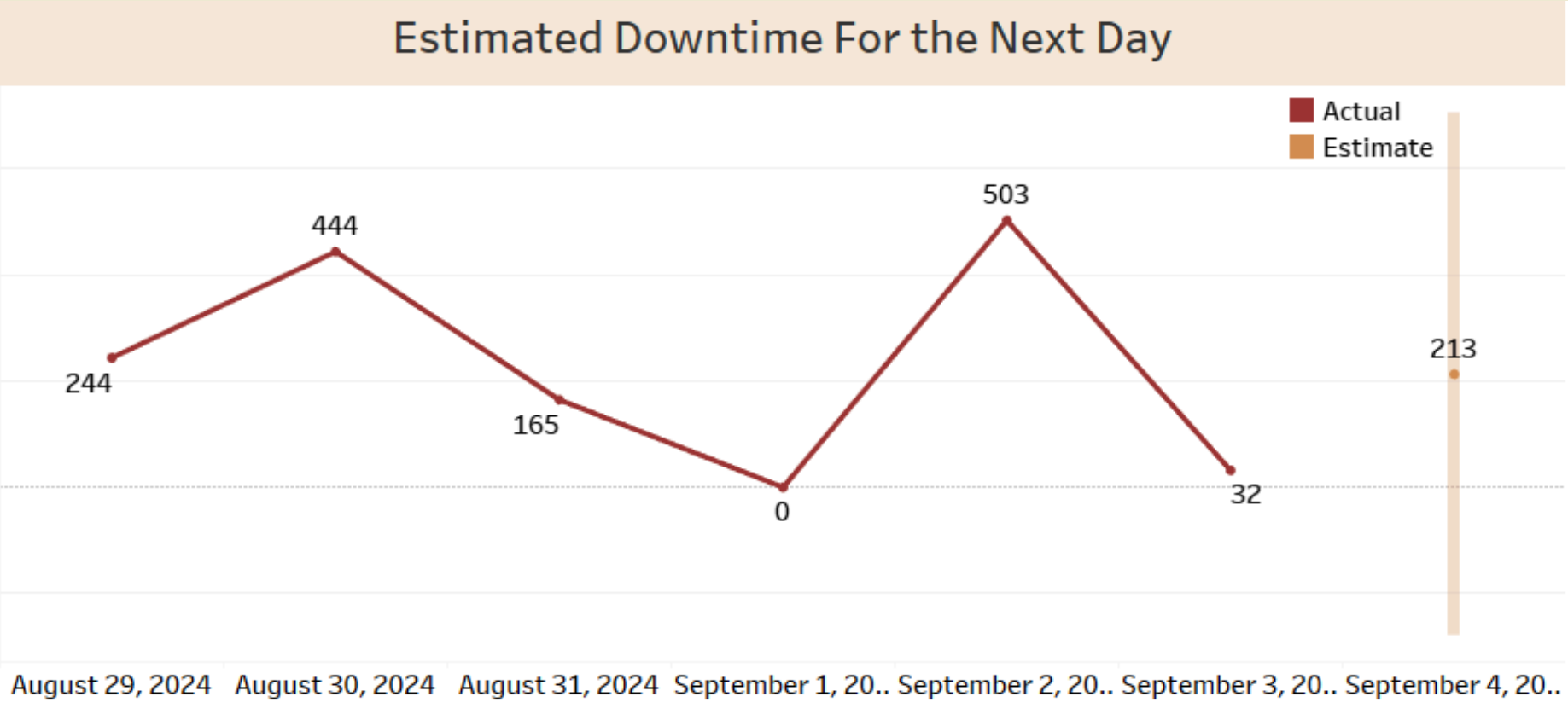
# Visualization
forecast_dates = pd.date_range(start=daily_summary['Date'].max() + pd.Timedelta(days=1), periods=2)

# Extend actual data for plotting
plot_df = daily_summary.copy()
forecast_df = pd.DataFrame({
    'Date': forecast_dates,
    'total_downtime': downtime_forecast,
    'batch_count': batch_forecast
})

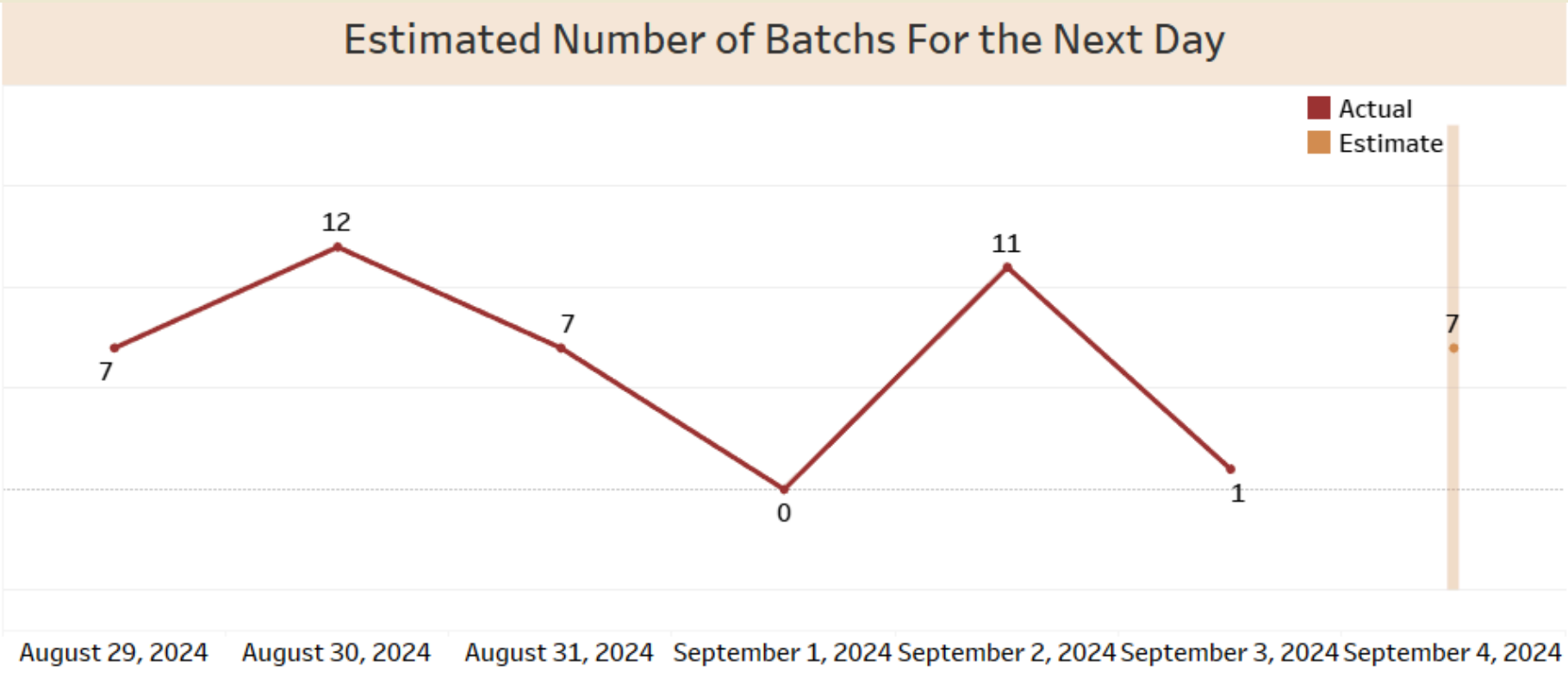
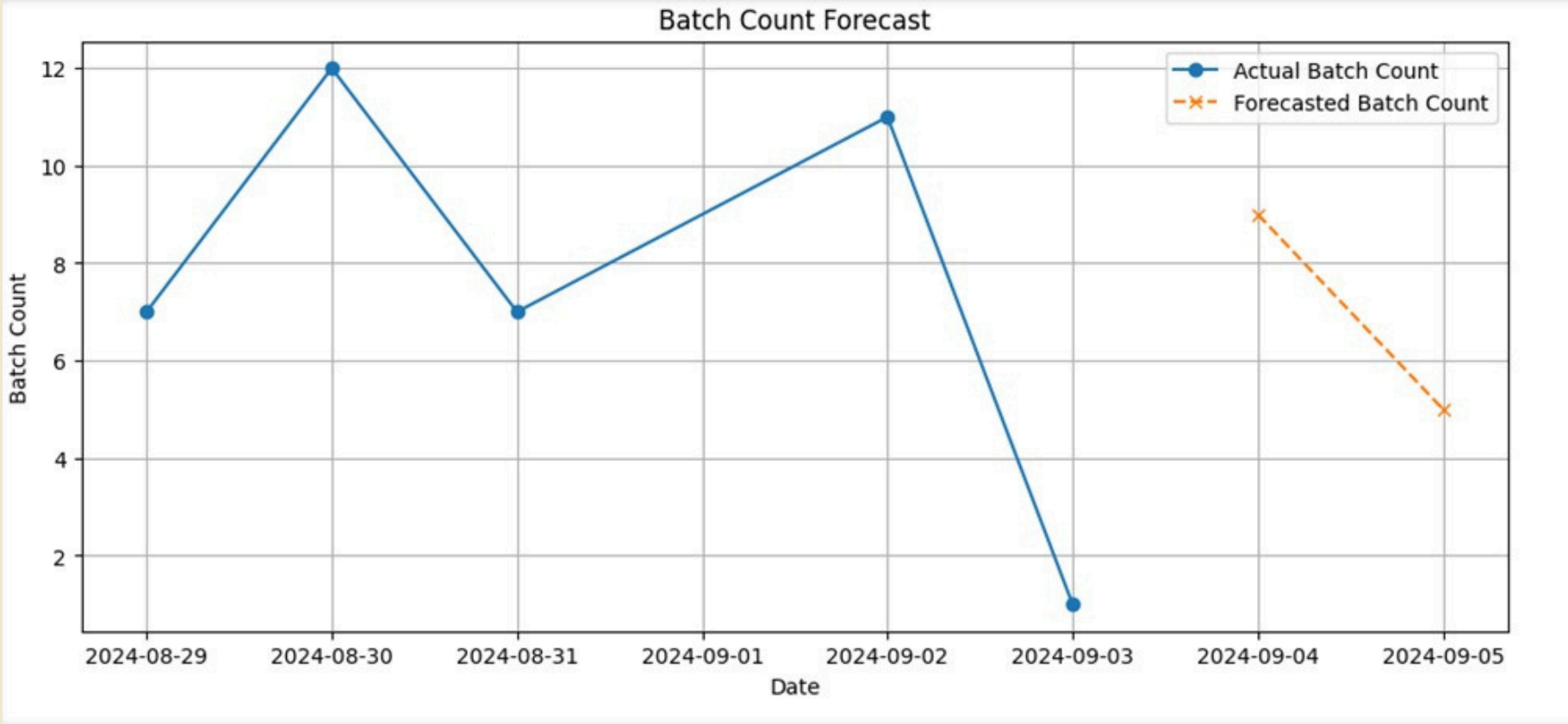
# Plot downtime
plt.figure(figsize=(12, 5))
plt.plot(plot_df['Date'], plot_df['total_downtime'], label='Actual Downtime', marker='o')
plt.plot(forecast_df['Date'], forecast_df['total_downtime'], label='Forecasted Downtime', marker='x', linestyle='--')
plt.title('Downtime Forecast')
plt.xlabel('Date')
plt.ylabel('Total Downtime (minutes)')
plt.legend()
plt.grid(True)
plt.show()

# Plot batch count
plt.figure(figsize=(12, 5))
plt.plot(plot_df['Date'], plot_df['batch_count'], label='Actual Batch Count', marker='o')
plt.plot(forecast_df['Date'], forecast_df['batch_count'], label='Forecasted Batch Count', marker='x', linestyle='--')
plt.title('Batch Count Forecast')
plt.xlabel('Date')
plt.ylabel('Batch Count')
plt.legend()
plt.grid(True)
plt.show()

Predicted Downtime (next 2 days): (399, 154)
Predicted Batch Count (next 2 days): (9, 5)
```



- The number of batches to be produced next day.



```
# Forecast next two days
def forecast_next_two(model, last_values):
    pred_day1 = model.predict([last_values])[0]
    pred_day2 = model.predict([[pred_day1] + list(last_values[:2])])[0]
    return round(pred_day1), round(pred_day2)

last_downtime_vals = daily_summary['total_downtime'].iloc[-3:].values[::-1]
downtime_forecast = forecast_next_two(downtime_model, last_downtime_vals)

last_batch_vals = daily_summary['batch_count'].iloc[-3:].values[::-1]
batch_forecast = forecast_next_two(batch_model, last_batch_vals)

print("Predicted Downtime (next 2 days):", downtime_forecast)
print("Predicted Batch Count (next 2 days):", batch_forecast)

# Visualization
forecast_dates = pd.date_range(start=daily_summary['Date'].max() + pd.Timedelta(days=1), periods=2)

# Extend actual data for plotting
plot_df = daily_summary.copy()
forecast_df = pd.DataFrame({
    'Date': forecast_dates,
    'total_downtime': downtime_forecast,
    'batch_count': batch_forecast
})

# Plot downtime
plt.figure(figsize=(12, 5))
plt.plot(plot_df['Date'], plot_df['total_downtime'], label='Actual Downtime', marker='o')
plt.plot(forecast_df['Date'], forecast_df['total_downtime'], label='Forecasted Downtime', marker='x', linestyle='--')
plt.title('Downtime Forecast')
plt.xlabel('Date')
plt.ylabel('Total Downtime (minutes)')
plt.legend()
plt.grid(True)
plt.show()

# Plot batch count
plt.figure(figsize=(12, 5))
plt.plot(plot_df['Date'], plot_df['batch_count'], label='Actual Batch Count', marker='o')
plt.plot(forecast_df['Date'], forecast_df['batch_count'], label='Forecasted Batch Count', marker='x', linestyle='--')
plt.title('Batch Count Forecast')
plt.xlabel('Date')
plt.ylabel('Batch Count')
plt.legend()
plt.grid(True)
plt.show()

Predicted Downtime (next 2 days): (399, 154)
Predicted Batch Count (next 2 days): (9, 5)
```

Introduction

Data Loading

Data Cleaning & Preprocessing

Data Transformation

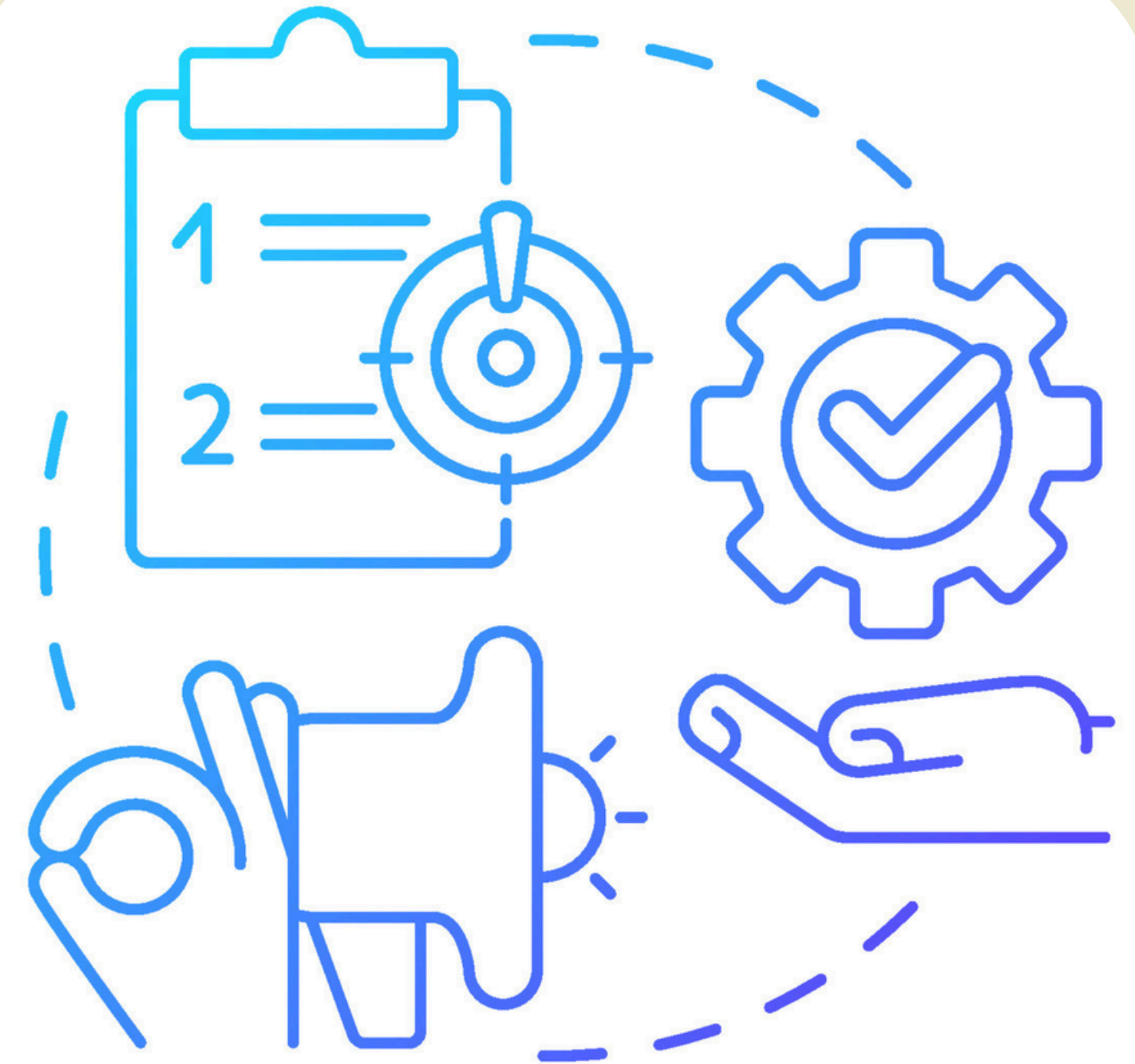
Data Modeling

Exploratory Data Analysis

Dashboards

Forecasting

✓ *Recommendations* ✓



RECOMMENDATIONS

1. Address Operator-Driven Downtime (56% of total)

a. Reduce Machine Adjustment Time (332 mins – 24%)

- **Recommendation:** Standardize machine settings with SOPs for each product.
- **Action:** Implement checklists or digital settings presets.
- **Impact:** Reduces variability and retraining needs.

b. Improve Training for Batch Changes & Coding (305 mins)

- **Recommendation:** Conduct targeted re-training for operators on batch changeover and coding protocols.
- **Action:** Develop quick-reference guides and conduct root cause reviews of past errors.
- **Impact:** Reduces avoidable errors and setup times.

2. Tackle Major Non-Operator Issues (44% of total)

a. Prevent Machine Failures (254 mins)

- **Recommendation:** Strengthen preventive maintenance schedules.
- **Action:** Use sensors/logs to predict failures and schedule timely maintenance.
- **Impact:** Minimizes unexpected downtime.

b. Resolve Inventory Shortages (225 mins)

- **Recommendation:** Integrate real-time inventory monitoring with alerts.
- **Action:** Implement a monitoring system or improve coordination with supply chain.
- **Impact:** Ensures materials are available before they run out.



Thanks & Acknowledgement



THANKYOU

We would like to extend our sincere gratitude to Eng. Ahmed for his continuous effort and dedication throughout the course.

His clear explanations, patience, and commitment to helping us understand the material made a significant difference in our learning experience.

We truly appreciate the time and energy he invested in us, and we wish him all the best and continued success in his career.

Thank you once again for everything!



We extend our deepest gratitude to:

- Digital Egypt Pioneers Initiative (DEPI).
- Ministry of Communications and Information Technology.

For their invaluable support, guidance, and collaboration in making this initiative a success.



